

سری دوم تمرین‌های بهینه‌سازی محدب

استاد: دکتر محسن هوشمند

دستار آموزش: آقای امیررضا عباسی

دانشجو: فائزه احمدی

۱۴۰۴۴۱۴۱

زمستان ۱۴۰۴

تابع زیر را با نقطه شروع‌های مختلف با روش‌های مختلف تعیین طول قدم (اعم از شرایط وولف و گلدشتاین، مومنتوم، مومنتوم نستروف، آداگراد، آدالتا، آراماس-پراپ، و آدام) را برای الگوریتم‌های شبه‌نیوتنی پیاده‌سازی و تحلیل کنید. نمودار تابع به علاوه قدم هریک و مقایسه آنها (تعداد گام‌ها و هزینه هر گام) رسم شود و تحلیل بر اساس آن گزارش شود. ضرایب مختلف را آزمایش کنید.

$$f(x) = (2 - x_1)^2 + 100(x_2 - x_1^2)^2$$

در این پروژه، من بیشتر با نسخه تغییر داده شده تابع روزن بروک با $a = 2$ آشنا شدم و به نسبت تمرین قبلی آگاهی بیشتری نسبت به هدف این تمرین و تابعی که آقای روزن بروک در دهه ۱۹۶۰ معرفی کرده، پیدا کردم. این تابع روزن بروک است:

$$f(x) = (2 - x_1)^2 + 100(x_2 - x_1^2)^2$$

پایین‌ترین نقطه این تابع (جایی که مقدار تابع صفر می‌شود) در مختصات $(2, 4)$ قرار دارد. خواسته سوال از ما این بود که این ۷ الگوریتم را در نقاط مختلفی از این تابع رها کنیم و ببینیم کدام یک از آنها قادر است سریع‌تر و با کمترین میزان محاسبات، راه خود را به سمت مینیمم سراسری پیدا کند.

برای توضیح ساده مسئله می‌توان این طور در نظر گرفت که ما می‌خواهیم پایین‌ترین مقدار یک تابع را پیدا کنیم. در عمل، تنها اطلاعاتی که در هر نقطه از تابع در اختیار داریم، شیب آن نقطه است. ما از درس یادگیری ماشین می‌دانیم که در ریاضیات به این شیب گرادیان گفته می‌شود. الگوریتم‌های مبتنی بر گرادیان از همین اطلاعات استفاده می‌کنند. یعنی در هر مرحله گرادیان تابع محاسبه می‌شود و سپس در جهتی حرکت می‌کنیم که مقدار تابع کمتر شود. با تکرار این فرایند، الگوریتم به تدریج به سمت نقطه‌ای حرکت می‌کند که مقدار تابع در آن حداقل است.

در این پروژه دو دسته از الگوریتم‌های بهینه‌سازی را بررسی کردم. این دسته بندی بر اساس روشی است که خودم برای فهم بهتر موضوع در نظر گرفتم و توضیح‌ها نیز بر همین اساس نوشته شده‌اند.

دسته اول الگوریتم‌های مرتبه اول هستند؛ مانند adam, momentum و rmsprop. این روش‌ها برای حرکت به سمت نقطه بهینه فقط از اطلاعات گرادیان (شیب تابع) استفاده می‌کنند. اندازه قدمی که در هر تکرار برداشته می‌شود به نرخ یادگیری بستگی دارد که ثابت یا تطبیقی است.

دسته دوم روش‌های شبه نیوتنی مانند ب‌ف‌گ‌ش هستند که همراه با جستجوی خطی استفاده می‌شوند. این روش‌ها علاوه بر گرادیان، سعی می‌کنند اطلاعاتی از انحنای تابع را نیز تخمین بزنند (از طریق تقریب ماتریس هسین). همچنین با استفاده از شرایط wolfe طول گام مناسب را در هر تکرار تعیین می‌کنند. به همین دلیل با تعداد تکرار کمتری به جواب نزدیک می‌شوند.

گفتم که تابعی که در این پروژه استفاده شده از خانواده توابع روزن بروک است. این تابع یکی از توابع شناخته شده برای آزمایش الگوریتم‌های بهینه‌سازی است. وجود ضریب ۱۰۰ در فرمول باعث می‌شود شکل تابع یک دره بسیار باریک و خمیده داشته باشد. به همین دلیل پیدا کردن مسیر مناسب برای رسیدن به نقطه مینیمم ساده نیست.

در طول اجرای الگوریتم‌ها با چند مشکل اصلی رو به رو شدم:

- اول، رفتار زیگزاگی در روش‌های مرتبه اول. به دلیل شیب زیاد دیواره‌های تابع، اگر اندازه گام بزرگ باشد الگوریتم به جای حرکت در امتداد دره، بین دو سمت آن نوسان می‌کند.
- دوم، کند شدن حرکت در نزدیکی کف دره. در این ناحیه تغییرات تابع بسیار کوچک می‌شود و اگر نرخ یادگیری کوچک انتخاب شود، الگوریتم با وجود تعداد تکرار زیاد به‌سختی به نقطه مینیمم نزدیک می‌شود.
- سوم، سرریز عددی (overflow). در برخی نقاط که مقدار گرادیان بسیار بزرگ می‌شود، مقادیر محاسباتی از محدوده قابل نمایش خارج می‌شوند و اجرای برنامه با مشکل مواجه می‌شود.

مسائل بهینه‌سازی از همین نوع در بسیاری از کاربردهای یادگیری ماشین دیده می‌شوند. هنگام آموزش مدل‌های یادگیری عمیق نیز باید مقدار یک تابع خطا را کمینه کنیم. این توابع دارای نواحی باریک، شیب‌های تند و بخش‌های نسبتاً مسطح هستند. به همین دلیل رفتار الگوریتم‌های بهینه‌سازی در چنین توابعی اهمیت زیادی دارد.

در این پروژه تلاش کردم تفاوت رفتار چند الگوریتم مختلف را روی این تابع بررسی کنم. نتایج نشان می‌دهد که روش‌هایی مانند آدام با تنظیم مناسب نرخ یادگیری می‌توانند عملکرد پایدارتری داشته باشند، در حالی که روش ب‌ف‌گ‌ش با وجود محاسبات بیشتر در هر تکرار، در تعداد مراحل کمتری به جواب نزدیک می‌شود. در ادامه، نحوه پیاده‌سازی برنامه و نمودارهای به دست آمده را توضیح می‌دهم.

توضیح کد پروژه:

```
> optimization_algorithms_comparison.py > nesterov
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
```

در ابتدا کتابخانه‌های پایه‌ای که برای پیش بردن پروژه به آن‌ها نیاز داشتیم را به برنامه اضافه کردم. از کتابخانه نامپای برای انجام راحت‌تر محاسبات عددی، ریاضیاتی و کار با آرایه‌ها استفاده کردم، چون در این الگوریتم‌ها دائما با بردارها و گرادیان‌ها سر و کار داریم. در کنار آن، برای اینکه در نهایت بتوانم خروجی‌های گرافیکی، شکل دره مانند تابع و مسیر حرکت تک تک الگوریتم‌ها را به صورت بصری رسم کنم، کتابخانه مت پلات لیب را وارد برنامه کردم. در نهایت هم برای پیاده‌سازی روش شبه نیوتنی یا همان ب‌ف‌گ‌ش، از تابع مینیمایز در کتابخانه سایپای کمک گرفتم تا بتوانم این روش پیشرفته را بهینه‌تر و دقیق‌تر روی تابع اجرا کنم.

```
# تابع و گرادیان
def f(x):
    return (2 - x[0])**2 + 100 * (x[1] - x[0]**2)**2

def grad_f(x):
    df_dx1 = -2*(2 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
    df_dx2 = 200*(x[1] - x[0]**2)
    return np.array([df_dx1, df_dx2])
```

در قدم بعدی، باید تابعی که قرار بود الگوریتم‌ها را روی آن محک بزنم تعریف می‌کردم. برای این کار فرمول ریاضی تابع هدف را در قالب یک تابع پایتون نوشتم که یک نسخه از تابع معروف روزن بروک است و به خاطر شکل دره مانند و چالش برانگیزش که توضیح دادم برای این پروژه انتخاب شده است. اما از آنجایی که بیشتر الگوریتم‌های بهینه سازی ما برای پیدا کردن مسیر درست نیاز دارند شیب را بدانند، تابع دوم را برای محاسبه گرادیان نوشتم. در این قسمت مشتقات جزئی تابع اصلی را از نظر ریاضی نسبت به متغیرهای اول و دوم حساب کردم و فرمول‌هایشان را قرار دادم تا الگوریتم‌ها با دریافت این جهت‌ها متوجه شوند که از کدام سمت باید به سمت کف دره و نقطه مینیمم حرکت کنند. در آخر هم این شیب‌ها را در قالب یک آرایه نامپای به عنوان خروجی تابع در نظر گرفتم تا استفاده از آن‌ها در محاسبات بعدی راحت باشد.

```
# پیاده سازی الگوریتم های مرتبه اول
def momentum(start, lr=0.001, beta=0.9, iters=1000):
    x = np.array(start, dtype=float)
    v = np.zeros_like(x)
    path = [x.copy()]
    for _ in range(iters):
        v = beta * v - lr * grad_f(x)
        x += v
        path.append(x.copy())
    return np.array(path), iters
```

در این مرحله اولین الگوریتم بهینه سازی، یعنی روش مومنتوم را پیاده سازی کردم. ایده اصلی این روش این است که الگوریتم فقط به شیب فعلی تابع نگاه نکند، بلکه اثر حرکت های قبلی را هم در نظر بگیرد. در کدی که نوشتیم، ابتدا نقطه شروع (start) مشخص می شود و متغیری به نام v با مقدار صفر تعریف کردم که نقش مومنتوم یا همان سرعت حرکت را دارد. همچنین یک لیست به نام `path` ساختم و نقطه شروع را در آن قرار دادم تا مسیر حرکت الگوریتم در طول اجرا ذخیره شود.

بعد از آن یک حلقه تکرار اجرا می شود. در هر مرحله، ابتدا مقدار قبلی v با ضریب `beta` ضرب می شود و سپس مقدار گرادیان نقطه فعلی (که در نرخ یادگیری `lr` ضرب شده) از آن کم می شود. به این ترتیب مقدار جدید v به دست می آید. در ادامه این مقدار به نقطه فعلی `x` اضافه می شود تا موقعیت جدید الگوریتم مشخص شود. این فرآیند در هر تکرار انجام می شود و نقطه جدید نیز در لیست `path` ذخیره می شود. در پایان نیز مسیر حرکت الگوریتم و تعداد تکرارها برگردانده می شود تا بتوان مسیر حرکت الگوریتم را روی نمودار رسم و بررسی کرد.

```
def nesterov(start, lr=0.001, beta=0.9, iters=1000):
    x = np.array(start, dtype=float)
    v = np.zeros_like(x)
    path = [x.copy()]
    for _ in range(iters):
        x_lookahead = x + beta * v
        v = beta * v - lr * grad_f(x_lookahead)
        x += v
        path.append(x.copy())
    return np.array(path), iters
```

در این بخش نسخه پیشرفته تری از مومنتوم، یعنی الگوریتم نستروف را پیاده سازی کردم. تفاوت اصلی این روش با مومنتوم معمولی در این است که قبل از محاسبه گرادیان، کمی جلوتر را در نظر می گیرد.

در روش مومنتوم، گرادیان در همان نقطه فعلی محاسبه می شود. اما در الگوریتم نستروف ابتدا با استفاده از مومنتوم تخمین می زنیم که اگر همین روند ادامه پیدا کند، در قدم بعدی تقریباً به چه نقطه ای می رسیم. سپس گرادیان را در آن نقطه تخمینی محاسبه می کنیم. به این ترتیب الگوریتم اطلاعات بهتری از مسیر پیش رو دارد و می تواند جهت حرکت را دقیق تر تنظیم کند. همین موضوع باعث می شود در بسیاری از مسائل، نستروف سریع تر و پایدارتر از مومنتوم معمولی به سمت نقطه مینیمم حرکت کند. در کد من، روال اولیه دقیقاً شبیه تابع قبل

است؛ نقطه شروع (start) را تنظیم کردم، متغیر سرعت (v) را با صفر مقداردهی اولیه کردم و لیست path را برای ثبت ردپای الگوریتم ساختم.

اما در داخل حلقه تکرار، ابتدا خط مهم و متفاوت $x_lookahead = x + \beta v$ را نوشتم. این متغیر در واقع مختصات همان نقطه پیش بینی شده در آینده را می سازد. سپس به جای اینکه تابع محاسبه گرادیان را برای موقعیت فعلی صدا بزنم، شیب را برای نقطه آینده $x_lookahead$ محاسبه کردم. در نهایت، سرعت v و موقعیت اصلی الگوریتم x را با استفاده از این گرادیان بهتر به روزرسانی کردم و مختصات جدید را به لیست مسیر اضافه کردم.

این ویژگی پیش بینی باعث می شود وقتی الگوریتم با سرعت بالا در حال حرکت است و در آستانه رسیدن به یک سربالایی یا تغییر شیب ناگهانی قرار دارد، بتواند سریع تر ترمز کند و جهت خود را تغییر دهد. این کار باعث می شود الگوریتم از نقطه مینیمم رد نشود و نوسانات بسیار کمتری نسبت به مومنتوم ساده داشته باشد. در آخر هم مانند توابع قبلی، مسیر طی شده و تعداد دفعات تکرار را به عنوان خروجی برگرداندم.

```
def adagrad(start, lr=0.5, epsilon=1e-8, iters=1000):
    x = np.array(start, dtype=float)
    G = np.zeros_like(x)
    path = [x.copy()]
    for _ in range(iters):
        g = grad_f(x)
        G += g**2
        x -= lr / (np.sqrt(G) + epsilon) * g
        path.append(x.copy())
    return np.array(path), iters
```

در این مرحله، به سراغ پیاده سازی الگوریتم متفاوتی به نام آداگراد رفتیم. ایده اصلی این روش، برخلاف الگوریتم های قبلی، شخصی سازی و تنظیم خودکار نرخ یادگیری برای تک تک متغیرها و جهتهاست. فرض کنید در حال حرکت در دره ای هستیم که در یک جهت (مثلا محور عمودی) شیب های بسیار تند و نوسانات شدیدی دارد، اما در جهت دیگر (محور افقی) بسیار مسطح و کم تغییر است. آداگراد به گونه ای عمل می کند که در جهت های پرنوسان و شیب دار، قدم های محتاطانه و کوچکی برمی دارد (انگار ترمز می گیرد) و در جهت های مسطح که تغییرات کمی داشته اند، گام های بزرگتری برمی دارد تا سریع تر پیش برود و متوقف نشود.

در کدی که برای این روش نوشتم، بعد از دریافت نقطه شروع و ساخت لیست path برای ثبت مسیر، یک متغیر مهم به نام G تعریف کردم که در ابتدا مقادیر آن صفر است. این متغیر در واقع نقش حافظه ی تاریخی شیب ها را برای الگوریتم بازی می کند.

در داخل حلقه تکرار، ابتدا گرادیان یا همان شیب فعلی g را محاسبه کردم. سپس در خط کلیدی $G += g^2$ ، مجذور (توان دوم) این گرادیان را حساب کرده و به متغیر G اضافه کردم. این یعنی هرچه در یک محور شیب‌های تندتری را تجربه کنیم، مقدار G برای آن محور به سرعت بزرگ و بزرگ‌تر می‌شود.

حالا به بخش جذاب ماجرا یعنی به‌روزرسانی موقعیت x می‌رسیم. در این خط، نرخ یادگیری اولیه (lr) را بر جذر همین حافظه‌ی انباشته‌شده (یعنی $\sqrt{G}$) تقسیم کردم. این ترفند ریاضی باعث می‌شود که به طور خودکار، برای متغیرهایی که G بزرگی دارند، نرخ یادگیری به شدت افت کند و الگوریتم در آن جهت‌ها گام‌های ریزی بردارد. همچنین یک مقدار بسیار کوچک به نام ϵ (مثلاً 10^{-8}) را به مخرج کسر اضافه کردم تا خیالم راحت باشد که اگر شیب‌ها صفر بودند، از نظر ریاضی با خطای تقسیم بر صفر مواجه نمی‌شوم. در نهایت، مانند توابع قبلی، موقعیت جدید را در لیست مسیر ذخیره کردم و در پایان کار، آرایه کامل مسیر طی شده و تعداد گام‌ها را به عنوان خروجی برگرداندم تا بتوانیم رفتار تطبیقی و هوشمندانه این الگوریتم را روی نمودار ببینیم.

```
def rmsprop(start, lr=0.1, beta=0.9, epsilon=1e-8, iters=1000):
    x = np.array(start, dtype=float)
    v = np.zeros_like(x)
    path = [x.copy()]
    for _ in range(iters):
        g = grad_f(x)
        v = beta * v + (1 - beta) * g**2
        x -= lr / (np.sqrt(v) + epsilon) * g
        path.append(x.copy())
    return np.array(path), iters
```

در این قسمت، نوبت به الگوریتم آر ام اس پراپ رسید. این روش برای حل بزرگ‌ترین مشکل الگوریتم قبلی (آداگراد) طراحی شده است. مشکل آداگراد این بود که با گذشت زمان، متغیر حافظه دائماً بزرگ و بزرگ‌تر می‌شد و در نتیجه نرخ یادگیری آنقدر افت می‌کرد که الگوریتم پیش از رسیدن به هدف، عملاً متوقف می‌شد.

برای رفع این مشکل، در این کد به جای جمع کردن تمام شیب‌های گذشته، از مفهوم میانگین متحرک نمایی استفاده کردم. در خط کلیدی $v = \beta v + (1 - \beta) g^2$ ، الگوریتم را طوری تنظیم کردم که حافظه گذشته (v) را با ضریب β کمی کم‌رنگ کند و فقط سهم کوچکی از مجذور شیب جدید g^2 را به آن اضافه کند. به این ترتیب، الگوریتم شیب‌های خیلی قدیمی را فراموش می‌کند و بیشتر به نوسانات اخیر توجه می‌کند.

در نهایت، موقعیت x را با تقسیم نرخ یادگیری بر جذر این متغیر تنظیم‌شده $\sqrt{v}$ به اضافه مقدار بسیار کوچک اپسیلون به‌روزرسانی کردم. این ترفند باعث می‌شود الگوریتم بدون اینکه متوقف شود، قدم‌هایش را در جهت‌های مختلف به صورت هوشمندانه و پیوسته تنظیم کند. در آخر هم مسیر حرکت برای رسم نمودار برگردانده می‌شود.

```
def adadelta(start, beta=0.9, epsilon=1e-8, iters=1000):
    x = np.array(start, dtype=float)
    E_g2 = np.zeros_like(x)
    E_dx2 = np.zeros_like(x)
    path = [x.copy()]
    for _ in range(iters):
        g = grad_f(x)
        E_g2 = beta * E_g2 + (1 - beta) * g**2
        dx = - (np.sqrt(E_dx2 + epsilon) / np.sqrt(E_g2 + epsilon)) * g
        E_dx2 = beta * E_dx2 + (1 - beta) * dx**2
        x += dx
        path.append(x.copy())
    return np.array(path), iters
```

در این قسمت، به سمت الگوریتم آدالدا رفتیم که در واقع نسخه پیشرفته‌تر و تکامل‌یافته آر ام اس پراپ است. جذاب‌ترین و مهم‌ترین ویژگی این الگوریتم که در همان خط اول کد هم خودنمایی می‌کند، حذف کامل نرخ یادگیری خوانده می‌شود. اگر به ورودی‌های تابع دقت کنید، می‌بینید که اصلاً متغیر η وجود ندارد. آدالدا آنقدر هوشمند طراحی شده که خودش به طور خودکار اندازه گام‌ها را تنظیم می‌کند و نیازی به تنظیم دستی نرخ یادگیری توسط ما ندارد. برای پیاده‌سازی این ایده، در کد به جای یک متغیر حافظه، از دو متغیر حافظه استفاده کردم:

1. متغیر E_{g2} : برای ذخیره میانگین متحرک مجذور گرادیان‌ها (دقیقاً مثل الگوریتم قبلی).

2. متغیر جدید E_{dx2} : برای ذخیره میانگین متحرک مجذور گام‌هایی (updates) که تا الان برداشته‌ایم.

در داخل حلقه، ابتدا متغیر اول (E_{g2}) را با گرادیان جدید به روز رسانی می‌کنیم. سپس در خط محاسبه گام dx ، به جای ضرب کردن گرادیان در یک نرخ یادگیری ثابت، آن را در کسری ضرب می‌کنیم که صورت آن ریشه حافظه گام‌ها $\sqrt{E_{dx2}}$ و مخرج آن ریشه حافظه شیب‌ها $\sqrt{E_{g2}}$ است. مقدار کوچک اپسیلون هم طبق معمول برای جلوگیری از خطای تقسیم بر صفر حضور دارد.

در نهایت، بلافاصله پس از محاسبه گام جدید dx ، حافظه گام‌ها E_{dx2} را با این مقدار جدید به‌روزرسانی می‌کنیم و موقعیت x را تغییر می‌دهیم. این رویکرد دوگانه باعث می‌شود واحد اندازه‌گیری گام‌ها از نظر ریاضی کاملاً با واحد متغیرها هم‌خوان شود و الگوریتم با تعادلی بی‌نظیر و بدون نیاز به حدس زدن نرخ یادگیری، به سمت هدف حرکت کند.


```
def adam(start, lr=0.1, beta1=0.9, beta2=0.999, epsilon=1e-8, iters=1000):
    x = np.array(start, dtype=float)
    m = np.zeros_like(x)
    v = np.zeros_like(x)
    path = [x.copy()]
    for t in range(1, iters + 1):
        g = grad_f(x)
        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * g**2
        m_hat = m / (1 - beta1**t)
        v_hat = v / (1 - beta2**t)
        x -= lr * m_hat / (np.sqrt(v_hat) + epsilon)
        path.append(x.copy())
    return np.array(path), iters
```

در این قسمت، به سراغ یکی از معروفترین، محبوبترین و قدرتمندترین الگوریتم‌های بهینه‌سازی در دنیای یادگیری ماشین، یعنی الگوریتم آدام رفتیم.

اگر بخواهیم آدام را ساده و همان‌طور که برای خودم معنی پیدا کرد توصیف کنم، باید بگوییم این الگوریتم بهترین‌های دو جهان را با هم ترکیب کرده است. یعنی همزمان هم ایده الگوریتم مونتوم (حفظ جهت و سرعت قبلی) و هم ایده الگوریتم آر ام اس پراپ (تطبیق دادن عاقلانه گام‌ها بر اساس شدت نوسانات) را در دل خود دارد. در کدی که نوشتیم، ابتدا دو متغیر حافظه تعریف کردم که هر دو در شروع کار صفر هستند:

1. متغیر m : نماینده مونتوم است که میانگین متحرک خود گرادیان‌ها را ذخیره می‌کند و جهت حرکت را مشخص می‌کند (با ضریب β_1).

2. متغیر v : نماینده بخش تطبیقی آر ام اس پراپ است که میانگین متحرک مجذور گرادیان‌ها را ذخیره می‌کند و شدت نوسانات را می‌سنجد (با ضریب β_2).

اما در داخل حلقه تکرار، علاوه بر به روزرسانی این دو متغیر با فرمول‌های آشنای قبلی، یک ترفند ریاضی بسیار مهم به نام تصحیح سوگیری (Bias Correction) را پیاده‌سازی کردم که در خطوط محاسبه‌ی m_hat و v_hat دیده می‌شود.

دلیل این کار چیست؟ از آنجایی که m و v در ابتدا با صفر مقداردهی می‌شوند، در قدم‌های اولیه‌ی الگوریتم به شدت تمایل دارند نزدیک به صفر بمانند (انگار موتور الگوریتم سرد است و اصطلاحاً سوگیری به سمت صفر دارند). با تقسیم کردن آن‌ها بر عبارت $(1 - \beta^t)$ (که t شماره‌ی مرحله فعلی است)، این سرد بودن اولیه را جبران می‌کنیم تا الگوریتم از همان گام اول با قدرت مسیرش را پیدا کند. با بزرگ شدن t در مراحل بعدی، اثر این مخرج کم کم از بین می‌رود.

در نهایت، در خط به روزرسانی موقعیت x ، الگوریتم را در جهت سرعت تصحیح‌شده m_hat حرکت دادیم، اما همزمان اندازه گام‌ها را با تقسیم بر جذر نوسانات v_hat کنترل کردم تا نرخ یادگیری به صورت کاملاً بهینه برای

هر جهت تنظیم شود. متغیر کوچک اپسیلون هم باز برای جلوگیری از خطای ریاضی حضور دارد. در آخر، مسیر قدرتمند آدام در لیست ذخیره و برگردانده می‌شود.

```
# تنظیمات و اجرای مقایسه‌ای
print("optimization algorithms simulator")
print("select a starting point:")
print("1. [-1.0, 2.0] (classic difficult start)")
print("2. [ 3.0, 3.0] (far start)")
print("3. [ 0.0, -1.0] (bottom of the valley)")

choice = input("enter 1, 2, or 3 (default is 1): ")
if choice == '2':
    start = [3.0, 3.0]
elif choice == '3':
    start = [0.0, -1.0]
else:
    start = [-1.0, 2.0]
```

در این قسمت یک منوی تعاملی ساده نوشتیم تا کاربر بتواند نقطه شروع حرکت الگوریتم‌ها را خودش انتخاب کند. چرا که در صورت سوال خواسته شده بود که با نقطه شروع‌های مختلف پیاده سازی را تست کنیم، این مشکلی بود که در تمرین آخر سری اول رعایتش نکردم.

برنامه سه نقطه با مختصات مختلف (با شرایط کلاسیک سخت، دور، یا پایین دره) را پیشنهاد می‌دهد و با دریافت ورودی کاربر از طریق دستور `input`، نقطه شروع (`start`) را مشخص می‌کند. با کمک شرط‌های `if` و `else`، اگر کاربر گزینه ۲ یا ۳ را انتخاب کند مختصات مربوط به آن‌ها تنظیم می‌شود و در غیر این صورت (یا در صورت وارد کردن مقدار نامعتبر)، برنامه به صورت پیش فرض همان گزینه اول را برای شروع در نظر می‌گیرد.

```
# گرفتن ضریب یادگیری پایه
user_lr = input("enter a base learning rate for adam/momentum (e.g. 0.01, 0.1) [default: 0.1]: ")
try:
    base_lr = float(user_lr)
except ValueError:
    base_lr = 0.1

print(f"\nrunning with start point: {start} and base learning rate: {base_lr}...\n")
```

در این بخش، برنامه از کاربر می‌خواهد مقدار پایه‌ای برای نرخ یادگیری را وارد کند. با استفاده از ساختار `try-except`، ورودی کاربر کنترل می‌شود: اگر کاربر یک عدد معتبر وارد کند، برنامه آن را به عنوان نرخ یادگیری می‌پذیرد. اما اگر ورودی نامعتبر باشد (مثلاً حرف وارد کند یا خالی بگذارد)، برنامه برای جلوگیری از توقف و خطا، به صورت خودکار مقدار پیش فرض 0.1 را جایگزین می‌کند. در نهایت، نقطه شروع و نرخ یادگیری انتخاب شده، برای تایید روی صفحه چاپ می‌شود تا کاربر بداند الگوریتم‌ها با چه تنظیماتی شروع به کار می‌کنند.

```

# برای آدام و تنظیم نسبی بقیه با base_lr استفاده از) اجرای الگوریتم ها
p_mom, cost_mom = momentum(start, lr=base_lr/200, iters=2000)
p_nes, cost_nes = nesterov(start, lr=base_lr/200, iters=2000)
p_ada, cost_ada = adagrad(start, lr=base_lr*5, iters=2000)
p_rms, cost_rms = rmsprop(start, lr=base_lr/2, iters=2000)
p_delta, cost_delta = adadelta(start, iters=2000) # نیاز ندارد lr آدالدا معمولا
p_adam, cost_adam = adam(start, lr=base_lr, iters=2000)

# استفاده میکند Wolfe که از جستجوی خطی با شرایط (BFGS) شبه نیوتنی
res_bfgs = minimize(f, start, method='BFGS', jac=grad_f, options={'return_all': True})
p_bfgs = np.array(res_bfgs.allvecs)
cost_bfgs = res_bfgs.nfev + res_bfgs.njev

```

در این بخش، تمام الگوریتم‌های بهینه‌سازی فراخوانی و اجرا می‌شوند تا مسیر حرکت (p) و هزینه محاسباتی (cost) هر کدام ذخیره شود. جزئیات این بخش به دو دسته تقسیم می‌شود:

1. الگوریتم‌های مرتبه اول (از مومنتوم تا آدام):

همه این متدها برای ۲۰۰۰ تکرار روی نقطه شروع انتخابی اجرا می‌شوند. نکته مهم، تغییراتی است که روی `base_lr` (نرخ یادگیری وارد شده توسط کاربر) برای هر الگوریتم اعمال شده است تا عملکردشان بهینه شود: آدام: مستقیما از همان `base_lr` استفاده می‌کند.

مومنتوم و نستروف: نرخ یادگیری آن‌ها به شدت کاهش یافته است (`base_lr/200`). از آنجا که این متدها سرعت را انباشته می‌کنند، در تابع روزن بروک (که شیب‌های تندی دارد) اگر نرخ یادگیری بالا باشد، اصطلاحا پرش‌های بزرگی می‌کنند و محاسباتشان منفجر (واگرا) می‌شود.

آداگراد: نرخ یادگیری آن ۵ برابر شده است (`base_lr*5`). در این روش مخرج کسر آپدیت وزن‌ها به سرعت بزرگ می‌شود و نرخ یادگیری را به شدت کاهش می‌دهد؛ بنابراین به یک مقدار اولیه بزرگتر نیاز دارد تا زود از حرکت باز نایستد.

آرام اس پراپ: نرخ یادگیری آن مقداری تعدیل و نصف شده است (`base_lr/2`).

آدالدا: نیازی به دریافت نرخ یادگیری اولیه ندارد؛ زیرا این الگوریتم به صورت خودکار اندازه گام‌ها را بر اساس میانگین متحرک تغییرات قبلی پارامترها تنظیم می‌کند.

2. الگوریتم شبه نیوتنی (بف‌گش):

در اینجا از تابع قدرتمند `minimize` از کتابخانه `scipy` استفاده شده است.

برخلاف الگوریتم‌های قبلی، روش ب‌ف‌گ‌ش نیازی به تنظیم دستی نرخ یادگیری ندارد. این روش از تکنیکی به نام جستجوی خطی (با رعایت شرایط wolfe) استفاده می‌کند تا در هر جهت حرکتی، بهترین و دقیق‌ترین اندازه گام را به صورت خودکار و پویا پیدا کند.

تنظیم `options={'return_all': True}` به کتابخانه می‌گوید که مختصات تمامی گام‌های طی شده را نگه دارد. این نقاط سپس در متغیر `p_bfgs` برای رسم مسیر در نمودار ذخیره می‌شوند.

در نهایت، هزینه الگوریتم ب‌ف‌گ‌ش (`cost_bfgs`) به عنوان مجموع دفعاتی که خود تابع (`nfev`) و گرادیان تابع (`njev`) محاسبه شده‌اند، به دست می‌آید. این یک معیار استاندارد برای سنجش بار محاسباتی در الگوریتم‌های بهینه‌سازی است.

```
# اجرای الگوریتم‌ها
p_mom, cost_mom = momentum(start, lr=0.0005, iters=2000)
p_nes, cost_nes = nesterov(start, lr=0.0005, iters=2000)
p_ada, cost_ada = adagrad(start, lr=0.5, iters=2000)
p_rms, cost_rms = rmsprop(start, lr=0.05, iters=2000)
p_delta, cost_delta = adadelta(start, iters=2000)
p_adam, cost_adam = adam(start, lr=0.1, iters=2000)

# استفاده می‌کند Wolfe که از جستجوی خطی یا شرایط (BFGS) شبه نیوتنی
res_bfgs = minimize(f, start, method='BFGS', jac=grad_f, options={'return_all': True})
p_bfgs = np.array(res_bfgs.allvecs)
cost_bfgs = res_bfgs.nfev + res_bfgs.njev
```

در این بخش، شش روش مختلف بهینه‌سازی (مومنتوم، نستروف، آداگراد، آر ام اس پراپ، آدالتا و آدام) به همراه یک روش پیشرفته‌تر اجرا می‌شوند. خلاصه کارکرد این کد به این شکل است:

1. روش‌های پایه: هر شش روش برای دو هزار مرحله اجرا می‌شوند. مقدار نرخ یادگیری (اندازه گام‌ها) برای هر روش به صورت ثابت و دستی تنظیم شده است. برای جلوگیری از خطای محاسباتی، این عدد برای روش‌هایی مثل مومنتوم بسیار کوچک، و برای آداگراد بزرگ‌تر در نظر گرفته شده است (آدالتا اصلاً نیازی به این عدد ندارد).

2. روش شبه نیوتنی: در انتها، این روش قدرتمند اجرا می‌شود که نیازی به تنظیم دستی اندازه گام‌ها ندارد و خودش مسیر را پیدا می‌کند. برنامه دستور می‌دهد که تمام نقاط مسیر طی شده ذخیره شوند. در پایان، هزینه محاسباتی این روش با جمع کردن تعداد دفعات محاسبه تابع و شیب آن به دست می‌آید.

```
# رسم نمودار
x1 = np.linspace(-2.5, 4.5, 400)
x2 = np.linspace(-1.5, 6.5, 400)
X1, X2 = np.meshgrid(x1, x2)
Z = (2 - X1)**2 + 100 * (X2 - X1**2)**2

fig, ax = plt.subplots(figsize=(12, 10))

# تنظیم پس‌زمینه صورتی کم‌رنگ
ax.set_facecolor(plt.rcParams['figure.facecolor'])

# رسم کانتورهای خطی
ax.contour(X1, X2, Z, levels=np.logspace(-1, 3.5, 30), cmap='viridis', alpha=0.6)
```

در این بخش، بوم نقاشی و نقشه پستی و بلندی‌ها برای نمایش حرکت الگوریتم‌ها آماده می‌شود. ابتدا یک محدوده مشخص برای طول و عرض تعیین شده و یک شبکه متراکم از نقاط ساخته می‌شود. سپس معادله تابع روی تمام این نقاط محاسبه می‌گردد تا شکل دره به دست آید. سپس یک صفحه برای رسم تصویر با ابعاد مشخص باز شده و رنگ پس‌زمینه آن به صورتی کم‌رنگ تغییر می‌یابد. در نهایت هم خطوط تراز (مانند نقشه‌های جغرافیایی که نقاط هم‌ارتفاع را به هم وصل می‌کنند) روی صفحه کشیده می‌شوند. این خطوط با کمی شفافیت و فاصله‌گذاری خاصی رسم می‌شوند تا شیب‌های تند و عمق دره به شکلی واضح و زیبا نمایش داده شوند.

```
# رسم مسیرها با استایل‌های مختلف برای وضوح بیشتر
ax.plot(p_mom[:, 0], p_mom[:, 1], '--', color='cyan', label=f'momentum (cost: {cost_mom})', linewidth=2)
ax.plot(p_nes[:, 0], p_nes[:, 1], '--', color='magenta', label=f'nesterov (cost: {cost_nes})', linewidth=2)
ax.plot(p_ada[:, 0], p_ada[:, 1], ':', color='lime', label=f'adagrad (cost: {cost_ada})', linewidth=2)
ax.plot(p_rms[:, 0], p_rms[:, 1], '--', color='orange', label=f'rmsprop (cost: {cost_rms})', linewidth=2)
ax.plot(p_delta[:, 0], p_delta[:, 1], '-', color='brown', label=f'adadelta (cost: {cost_delta})', linewidth=2)
ax.plot(p_adam[:, 0], p_adam[:, 1], '-', color='blue', label=f'adam (cost: {cost_adam})', linewidth=2.5)
ax.plot(p_bfgs[:, 0], p_bfgs[:, 1], '-o', color='black', label=f'quasi-newton + wolfe (cost: {cost_bfgs})', markersize=4, linewidth=2)

# نمایش نقطه کمینه سراسری با مثلث قرمز
ax.plot(2, 4, marker='^', color='red', markersize=16, markeredgecolor='black', label='global min (2,4)')
```

در این بخش، مسیر حرکت تک‌تک روش‌های جستجو روی نقشه دره رسم می‌شود. برای اینکه مسیرها با هم اشتباه گرفته نشوند، برای هر روش یک رنگ خاص و یک نوع خط متفاوت (مثل خط چین، نقطه چین یا خط ممتد) انتخاب شده است. همچنین هزینه محاسباتی هر روش در کنار نام آن درج شده تا در جدول راهنمای تصویر دیده شود. مسیر روش شبه نیوتنی با دایره‌های کوچک روی خط متمایز شده است. در ادامه، پایین‌ترین نقطه دره (کمینه سراسری) که هدف تمام این روش‌هاست، با یک مثلث بزرگ قرمز رنگ روی نقشه علامت‌گذاری می‌شود تا مشخص شود کدام روش توانسته دقیقاً به هدف برسد.

```
# تنظیمات گرافیکی نمودار
ax.set_title("comparison of optimization algorithms", fontsize=15, fontweight='bold', pad=15)
ax.set_xlabel("$x_1$", fontsize=12)
ax.set_ylabel("$x_2$", fontsize=12)
ax.legend(loc='upper right', framealpha=0.9, edgecolor='black')
ax.set_xlim(-2.5, 4.5)
ax.set_ylim(-1.5, 6.5)
ax.grid(color='white', linestyle='-', linewidth=1.5, alpha=0.8)

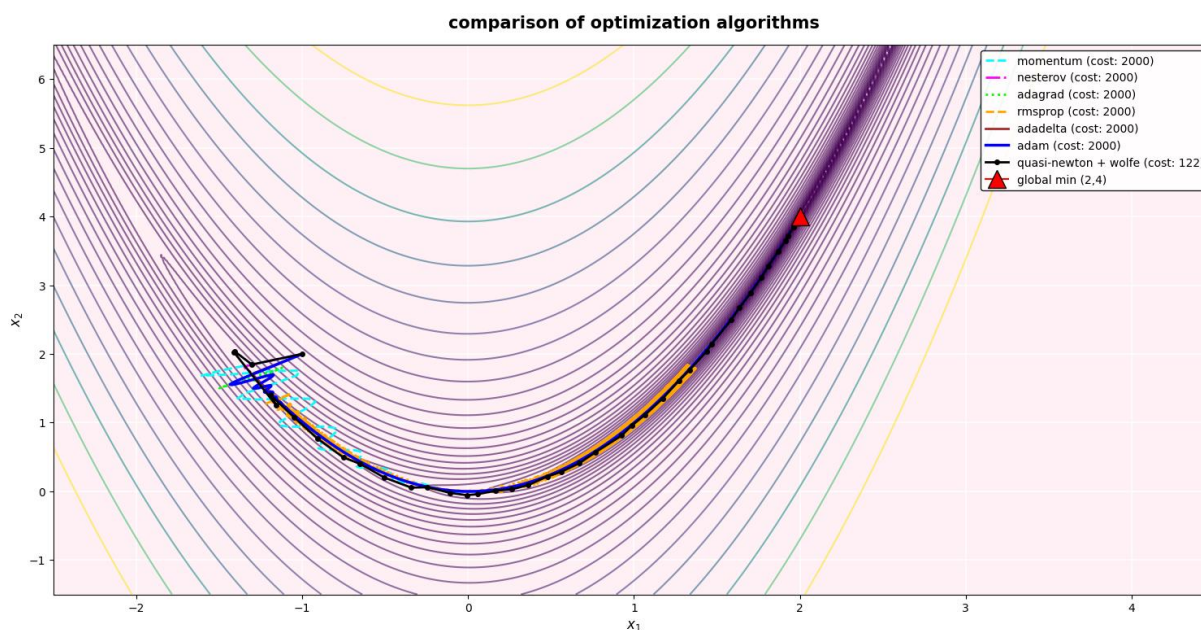
plt.tight_layout()
plt.show()

# چاپ مقایسه هزینه ها در ترمینال
print(f"cost (gradient/function evaluations):")
print(f"momentum: {cost_mom}, nesterov: {cost_nes}, adagrad: {cost_ada}")
print(f"RMSprop: {cost_rms}, adadelata: {cost_delta}, adam: {cost_adam}")
print(f"quasi-Newton(BFGS): {cost_bfgs}")
```

در قسمت نهایی کد، تصویر نهایی آراسته شده و نتایج عددی گزارش می‌شوند. مواردی مانند عنوان اصلی نمودار، نام محورهای افقی و عمودی، محدوده اندازه نقشه و خطوط شبکه‌ای پس زمینه تنظیم می‌شوند. سپس جدول راهنمای رنگ‌ها در گوشه تصویر قرار گرفته و نقشه نهایی به شما نمایش داده می‌شود. در آخر هم برنامه یک گزارش متنی در صفحه فرمان چاپ می‌کند که نشان می‌دهد هر روش، دقیقاً چند بار تابع و شیب آن را محاسبه کرده است. این اعداد بهترین معیار برای مقایسه سرعت و هزینه این روش‌ها با یکدیگر هستند.

تحلیل نمودارها

حالت اول: حرکت کند در کف دره



```

PS C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141> cd soal_1
PS C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1> python optimization_algorithms_comparison.py
optimization algorithms simulator
select a starting point:
1. [-1.0, 2.0] (classic difficult start)
2. [ 3.0, 3.0] (far start)
3. [ 0.0, -1.0] (bottom of the valley)
enter 1, 2, or 3 (default is 1): 1
enter a base learning rate for adam/momentum (e.g. 0.01, 0.1) [default: 0.1]: 0.1

running with start point: [-1.0, 2.0] and base learning rate: 0.1...

cost (gradient/function evaluations):
momentum: 2000, nesterov: 2000, adagrad: 2000
RMSprop: 2000, adadelta: 2000, adam: 2000
quasi-Newton(BFGS): 122

```

نقطه شروع $x = [-1.0, 2.0]$ و نرخ یادگیری را $\alpha = 0.1$ گذاشتیم. همانطور که در عکس خروجی کنسول یا ترمینال مشخص است، من برای تمام الگوریتم‌ها یک محدودیت گذاشتم که حداکثر ۲۰۰۰ بار شیب محاسبه می‌شود و هزینه یا $cost = 2000$ است. حالا هر ۶ روش مرتبه اول (آدام، مومنتوم، آر ام اس پراپ و...) عدد ۲۰۰۰ را نشان می‌دهند. این یعنی آن‌ها ۲۰۰۰ قدم برداشتند و سقف مجازشان تمام شد، اما هنوز به نقطه پایان نرسیدند. در مقابل، روش شبه نیوتنی (BFGS) با یک اختلاف فاحش، تنها با ۱۲۲ قدم موفق شد کار را تمام کند و به نقطه هدف برسد. حالا اگر به عکس نمودار نگاه کنیم، دقیقاً می‌فهمیم چرا آن ۶ الگوریتم در راه ماندند و چرا بفگش پیروز شد.

رفتار روش‌های مرتبه اول (خطوط رنگی درهم تنیده):

من الگوریتم‌ها را در نقطه $[-1.0, 2.0]$ یعنی جایی روی دیواره شیب دار دره رها کردم. خطوط رنگی (مثل خط آبی آدام و فیروزه‌ای مومنتوم) در ابتدا به دلیل شیب زیاد دیواره، با کمی نوسان و پرش زیگزاگ خودشان را به کف دره رساندند.

اما مشکل اصلی این تابع عجیب اینجا خودش را نشان داد که کف این دره بسیار مسطح و طولانی است. وقتی این الگوریتم‌ها به کف دره رسیدند، شیب زیر پایشان تقریباً صفر شد. چون من نرخ یادگیری را روی عدد کوچک ۰.۱ تنظیم کرده بودم، قدم‌های این الگوریتم‌ها در این منطقه مسطح به شدت ریز و میلی متری شد. آن‌ها مثل ماشین‌هایی که در گل گیر کرده باشند، زور می‌زدند ولی خیلی کند جلو می‌رفتند. در نهایت، همان طور که در تصویر می‌بینید، این خطوط رنگی در میانه راه و قبل از رسیدن به مثلث قرمز رنگ (مینیمم سراسری در نقطه ۲،۴)، فرصت ۲۰۰۰ قدمشان تمام شد و همانجا متوقف شدند.

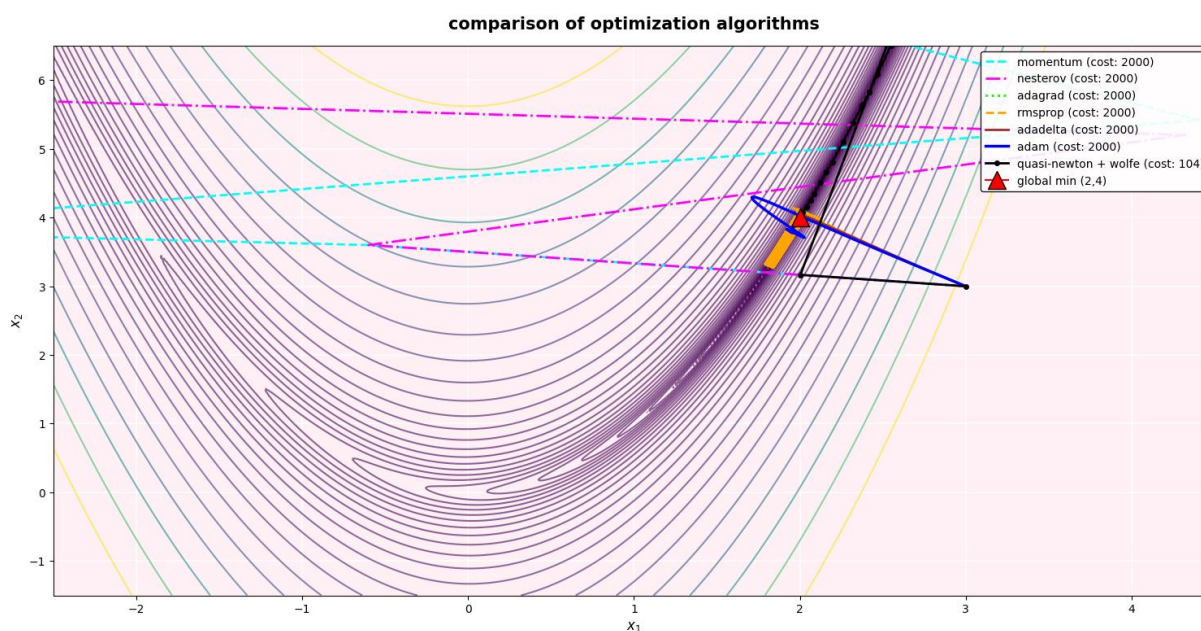
رفتار روش شبه نیوتنی – بفگش (خط مشکی رنگ):

الگوریتم بفگش (خط مشکی با نقطه‌های درشت) رفتار کاملاً متفاوتی داشت. این الگوریتم در ابتدا با چند گام بلند و مطمئن خودش را به کف دره رساند. سپس، وقتی به منطقه مسطح رسید، برخلاف روش‌های دیگر که قدم‌هایشان ریز شد، این الگوریتم به لطف استفاده از جستجوی خطی (شرایط wolfe) در هر قدم بررسی کرد

که: «مسیر جلوم تا کجا باز است؟». به همین دلیل، توانست با گام‌هایی بلند، منحنی موزی شکل دره را با زیبایی هرچه تمام‌تر طی کند و بدون اتلاف وقت، تنها با ۱۲۲ ارزیابی، دقیقاً روی هدف (مثلث قرمز) بنشیند.

این اجرا به من ثابت کرد که در توابعی که دره‌های طولانی و مسطح دارند، داشتن یک نرخ یادگیری ثابت (یا تطبیقی با پایه نامناسب) باعث کندی شدید الگوریتم خواهد شد. الگوریتم‌های مرتبه اول در این مسیرهای طولانی کم می‌آورند، در حالی که روش‌های شبه نیوتنی با تنظیم خودکار طول گام‌هایشان، برنده واقعی این میدان هستند.

حالت دوم: نوسان زیگزاگ و پرش از دیوار




```

PS C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1> python optimization_algorithms_comparison.py
optimization algorithms simulator
select a starting point:
1. [-1.0, 2.0] (classic difficult start)
2. [ 3.0, 3.0] (far start)
3. [ 0.0, -1.0] (bottom of the valley)
enter 1, 2, or 3 (default is 1): 2
enter a base learning rate for adam/momentum (e.g. 0.01, 0.1) [default: 0.1]: 0.5
running with start point: [3.0, 3.0] and base learning rate: 0.5...

C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:10: RuntimeWarning: overflow encountered in scalar power
  df_dx1 = -2*(2 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:11: RuntimeWarning: overflow encountered in scalar power
  df_dx2 = 200*(x[1] - x[0]**2)
C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:10: RuntimeWarning: invalid value encountered in scalar subtract
  df_dx1 = -2*(2 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:11: RuntimeWarning: invalid value encountered in scalar subtract
  df_dx2 = 200*(x[1] - x[0]**2)
C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:10: RuntimeWarning: overflow encountered in scalar multiply
  df_dx1 = -2*(2 - x[0]) - 400*x[0]*(x[1] - x[0]**2)
C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:31: RuntimeWarning: invalid value encountered in subtract
  v = beta * v - lr * grad_f(x_lookahead)
C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1\optimization_algorithms_comparison.py:20: RuntimeWarning: invalid value encountered in subtract
  v = beta * v - lr * grad_f(x)
cost (gradient/function evaluations):
momentum: 2000, nesterov: 2000, adagrad: 2000
RMSprop: 2000, adadelta: 2000, adam: 2000
quasi-Newton(BFGS): 104

```

نقطه شروع سمت چپ دره $x = [3.0, 3.0]$ و نرخ یادگیری $\alpha = 0.5$ قرار داده شد. اگر در سناریوی اول مشکل ما گُند بودن و گیر کردن در کفه دره بود، در این سناریو انگار مشکل ما انفجار و پرت شدن از روی کوه است. بیایید ببینیم چه اتفاقی افتاد. زمانی که من کلی هشدار در ترمینال گرفتم، فکر کردم باز هم اشتباه کرده‌ام و باید مشکل جدیدی را حل کنم، اما بعد فهمیدم که این بخشی از کار الگوریتم‌ها و برنامه من است.

چرا در ترمینال خطاهای هشدار زیادی مشاهده می‌شوند؟

گفتم که وقتی این حالت را اجرا کردم، ترمینال پر از هشدارهایی مثل `RuntimeWarning: overflow` (سرریز شدن) و `invalid value` (مقدار نامعتبر) شد. شاید در نگاه اول به نظر برسد کد خراب شده، اما بعد فهمیدم که این یک پدیده فوق‌العاده مهم در بهینه‌سازی محسوب می‌شود.

من الگوریتم‌ها را روی یک شیب خیلی تند رها کردم و به آن‌ها گفتم با سرعت و گام‌های بزرگ (نرخ یادگیری ۰.۵) حرکت کنید. در همان قدم اول، شیب زیر پایشان (گرادیان) عدد بسیار بزرگی بود. وقتی این عدد بزرگ در نرخ یادگیری بزرگ ضرب شد، الگوریتم‌ها تصمیم گرفتند یک پرش چند هزار کیلومتری انجام دهند.

وقتی کامپیوتر خواست موقعیت جدید آن‌ها را حساب کند، اعداد آنقدر بزرگ شدند که از حد تحمل حافظه ماشین حساب کامپیوتر (محدودیت متغیرهای `Float`) خارج شدند. به این حالت در ریاضیات سرریز عددی (`numerical overflow`) و در یادگیری ماشین انفجار گرادیان (`exploding gradients`) می‌گویند. در

نهایت، چون کامپیوتر نتوانست عدد را درک کند، مقدار متغیرها را NaN (مخفف Not a Number) قرار داد و الگوریتم‌ها از کار افتادند و با هزینه 2000 متوقف شدند.

اتفاقات داخل ترمینال، روی نمودار دقیقاً به شکل زیر تصویر کشیده شده‌اند:

شلیک به خارج از نقشه (خطوط فیروزه‌ای و ارغوانی): نگاه کنید که روش‌های کلاسیک مثل مومنتوم و نستروف چطور به صورت خطوط صاف به سمت راست و بالای تصویر شلیک شده‌اند. آن‌ها با دیدن شیب تند و داشتن نرخ یادگیری بالا، در همان گام اول از دیواره دره به بیرون پرت شدند و کلاً از نقشه و فضای مسئله خارج شدند (واگرایی مطلق).

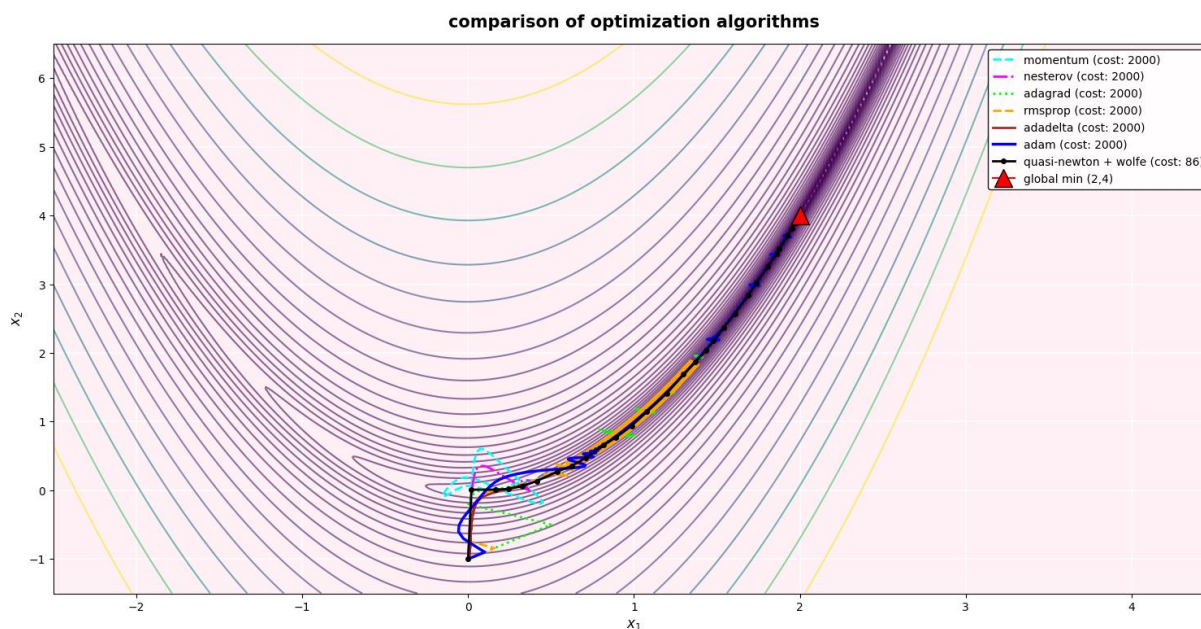
باقی مانند الگوریتم ولی انگار که گیج شده (خط آبی - آدام): الگوریتم آدام یک ویژگی ذاتی به نام نرخ یادگیری تطبیقی دارد؛ یعنی مثل یک کمک فدر عمل می‌کند و اثر شیب‌های خیلی وحشتناک را کمی خنثی می‌کند. به همین دلیل، آدام برخلاف مومنتوم به بیرون از نقشه پرت نشد و توانست خودش را به نزدیکی مثلث قرمز (نقطه هدف) برساند. اما چون نرخ یادگیری پایه را 0.5 گذاشته بودم، وقتی به نزدیکی هدف رسید، گام‌هایش برای آن منطقه خیلی بزرگ بود. در نتیجه شروع کرد به پرش‌های وحشتناک از این طرف به آن طرف نقطه هدف (به خط آبی ضخیم و در هم تنیده در کنار مثلث قرمز دقت کنید). او زنده ماند، اما نتوانست دقیقاً روی هدف بنشیند.

اقتدار و هوشمندی مطلق (خط مشکی - بف‌گش): حالا به شاهکار روش شبه نیوتنی نگاه کنید. این روش در کمال آرامش تنها با ۱۰۴ قدم به هدف رسیده است. چرا خطای overflow نداد؟ چرا پرت نشد؟

پاسخ این است که الگوریتم بف‌گش اصلاً به نرخ یادگیری دستی من $\alpha = 0.5$ اهمیت نمی‌دهد. این الگوریتم قبل از هر قدم، با استفاده از الگوریتم جستجوی خطی wolfe، شرایط را می‌سنجد و خودش محاسبه می‌کند که دقیقاً چه گامی بردارد که نه آن قدر بزرگ باشد که پرت شود و نه آنقدر کوچک که گیر بیفتد. او با دیدن شیب تند، در ابتدا یک پرش دقیق به سمت راست نقطه [1.7, 4] انجام داد و سپس با یک مسیر کاملاً مستقیم و باثبات به مینیمم سراسری رسید.

این اجرا به زیبایی ثابت کرد که روش‌های مرتبه اول (گرادیان نزولی و خانواده‌اش) به شدت بی‌دفاع و حساس به تنظیمات پارامترها (hyperparameters) هستند. یک انتخاب اشتباه در نرخ یادگیری پتانسیل این را دارد که کل فرآیند را نابود کند. اما خانواده روش‌های شبه نیوتنی به دلیل محاسبه خودکار طول گام، به شدت مقاوم هستند و تحت تاثیر شیب‌های تند و تنظیمات اشتباه ما قرار نمی‌گیرند.

حالت سوم: واگرایی شدید در برابر پایداری



```
PS C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1> python optimization_algorithms_comparison.py
● optimization algorithms simulator
select a starting point:
1. [-1.0, 2.0] (classic difficult start)
2. [ 3.0, 3.0] (far start)
3. [ 0.0, -1.0] (bottom of the valley)
enter 1, 2, or 3 (default is 1): 3
enter a base learning rate for adam/momentum (e.g. 0.01, 0.1) [default: 0.1]: 0.01

running with start point: [0.0, -1.0] and base learning rate: 0.01...

cost (gradient/function evaluations):
momentum: 2000, nesterov: 2000, adagrad: 2000
RMSprop: 2000, adadelata: 2000, adam: 2000
quasi-Newton(BFGS): 86
○ PS C:\Users\Faeze\Desktop\CO_2_Faeze_Ahmadi_14044141\soal_1>
```

همان‌طور که مشاهده می‌شود، نقطه شروع $x = [0.0, -1.0]$ و نرخ یادگیری را 0.01 در نظرم گرفتم که بسیار کوچک است. با نگاه به تصویر ترمینال، یک الگو بسیار جلب توجه می‌کند.

- توقف به دلیل اتمام زمان (Timeout): روبروی نام تمام الگوریتم‌های مرتبه اول (آدام، مومنتوم، آر ام اس پراپ و...) عدد ۲۰۰۰ نوشته شده است. در اینجا واگرایی یا خطای محاسباتی (مانند سناریوی دوم) رخ نداده است؛ بلکه این الگوریتم‌ها تمام ۲۰۰۰ گام مجاز خود را مصرف کرده‌اند، اما آن قدر کند بوده‌اند که پیش از رسیدن به هدف، ظرفیتشان تکمیل شده و متوقف شده‌اند.
- کارایی خیره‌کننده ب‌ف‌گ‌ش: در پایین تصویر، روش شبه نیوتونی ب‌ف‌گ‌ش با عدد ۸۶ خودنمایی می‌کند. این نشان می‌دهد که این روش نه تنها تحت تاثیر شیب کم و تنظیمات ما قرار نگرفته، بلکه با بهینه‌ترین حالت ممکن به پایان رسیده است.

تصویر نمودار به شکلی بی نظیر اتفاقات داخل ترمینال را تصویرسازی می‌کند:

- ترافیک سنگین در ابتدای مسیر (خطوط رنگی): به نقطه شروع در پایین تصویر (0,-1) نگاه کنید. خطوط رنگی مربوط به الگوریتم‌های گرادیانی (مثل آبی، فیروزه‌ای و سبز) به جای اینکه به سمت مثلث قرمز پرواز کنند، به شکل یک کلاف درهم تنیده و بسیار فشرده درآمده‌اند. دلیل این اتفاق، فرمول پایه این روش‌ها است:

$$\text{step} = \text{learning rate} \times \text{gradient}$$

در این نقطه، هم شیب تابع گرادیان بسیار کم است (چون در کف دره هستیم) و هم ما نرخ یادگیری را روی عدد بسیار کوچک ۰.۰۱ تنظیم کرده‌ایم. حاصل ضرب دو عدد کوچک، مساوی است با یک گام بسیار کوچکتر. این الگوریتم‌ها مسیر را درست تشخیص داده‌اند، اما با گام‌های میلی متری حرکت می‌کنند. بعد از ۲۰۰۰ قدم، آن‌ها تازه به میانه راه رسیده‌اند.

- گام‌های عاقلانه و منعطف (خط مشکی BFGS): حالا به مسیر خط مشکی رنگ دقت کنید. الگوریتم ب‌ف‌گ‌ش نیازی به نرخ یادگیری دستی ما ندارد. او با استفاده از تقریب ماتریس هسین متوجه می‌شود که در منطقه مسطحی قرار دارد و انحنا کم است. بنابراین، با کمک جستجوی خطی، متوجه می‌شود که با خیال راحت گام‌های بلندی در امتداد دره برمی‌دارد. نقطه‌های روی خط مشکی نشان دهنده گام‌های این الگوریتم هستند؛ همان‌طور که می‌بینید فاصله‌ی این نقاط از هم زیاد است و الگوریتم با اعتماد به نفس کامل و با سرعت، در ۸۶ قدم به مینیمم سراسری (مثلث قرمز) رسیده است.

این خروجی یک درس بزرگ در بهینه‌سازی شبکه‌های عصبی و یادگیری ماشین است. نرخ یادگیری بسیار کوچک همیشه ایمن نیست؛ بلکه مواردی چون مرگ خاموش الگوریتم (همگرایی بیش از حد کُند) را نیز به ارمغان می‌آورد. روش‌های مبتنی بر گرادیان در مناطق مسطح به شدت آسیب‌پذیرند، در حالی که روش‌های شبه نیوتنی با درک هندسه فضا، خود را با شرایط تطبیق داده و به سرعت از این مناطق عبور می‌کنند.

تجربه این تمرین به ما نشان می‌دهد که پیدا کردن پایین‌ترین نقطه در یک مسیر پر از پستی و بلندی، اصلاً کار ساده‌ای نیست. روش‌های معمولی با اینکه ساده‌تر هستند، در شیب‌های تند یا مسیرهای خیلی صاف به شدت گیج میشوند، قدم‌های اشتباه برمی‌دارند و دائماً نیاز دارند که ما اندازه قدم‌هایشان را با دقت برایشان تنظیم کنیم. در مقابل، روش‌های بهتر (مثل روش شبه نیوتنی) با هوشمندی بسیار بیشتر و بدون نیاز به دخالت و راهنمایی ما، مسیر درست را تشخیص داده و با صرف انرژی بسیار کمتر (محاسبات کمتر) مستقیماً به هدف می‌رسند؛ این یعنی در مسائل پیچیده، استفاده از ریاضیات کمی سنگین‌تر به شدت در زمان و هزینه صرفه جویی خواهد کرد.

2- الف) درستی کلمات روسی هاسی که نیوتونی را بسنجید. ب) معادله خانواده بر روی من را داده و

فهی اسکان کشتی و کجایی متناظر آن را به دقت و دقتی بسنجید. جی نشان دهنده بعضی

از روسی های معرفی شده که نیوتونی با مبحث پیکرهای مطرح شده در بیان مندرج در ارتباط هستند.

ابتدا تعاریف کوتاه از جیتی مسأله که برابر خود من در ابتدا تلف بود و لازم بود به جنبه

دکتر رجوی گفتم، آن کار را مطالعه کنم را بیان می کنم. در این ترتیب در ادامه مبنای بر جیتی تعاریف،

پایان سوال را می دهیم.

- روسی که نیوتونی چیست؟ در ریاضیات، برابر پیدا کردن کمترین مقدار (مینیم) یک تابع،

بهترین راه این است که علاوه بر یک تابع (مشتق اول یا بیان)، افتخاری تابع (مشتق دوم) یا

ماتریس هسین) را هم بدانیم. به این روشی که از مشتق دوم استفاده می کنند، روش نیوتونی هم روشی

اما مثل اینست که محاسبه دقیق ماتریس مشتق دوم برابر مسأله که متغیرهای زیادی دارند، برای

کامپیوتر بسیار دشوار و زمان بر است. در روش های که نیوتونی، ما مشتق دوم را به صورت دقیق

محاسبه نمی کنیم؛ بلکه همان طور که در مسیر پیدا کردن جواب همگام می کنیم، با بررسی تغییرات

مشتق اول در هم هاسی متوالی، یک تقریب از مشتق دوم (یا ماتریس هسین) می سازیم. این کار

سرعت محاسبات را به شدت بالا می برد.

- معادله خانواده بر روی من چیست؟ یک فرمول می ریاضی است که تمام روسی های مختلف تقریب زدن

مشتق دوم را در یک قالب یکپارچه با یک پارامتر متغیر (انتخاب) نشان می دهد.

- دقت چیست؟ محقق نام چهار دانشمند (Davidson, Fletcher, Russell) و یکی از اولین فرمول‌های
خانواده پدیدین برای تقریب زدن معکوس مشتق دوم است.

- بسف چیست؟ محقق نام چهار دانشمند (Bryden, Fletcher, Goldfarb, Shanno) و نویسنده معروف ترین و معروف ترین

فرمول شبه نیوتونی است. در عمل بسیار پایدارتر از دقت است و امروزه در اکثر نرم افزارها استفاده می‌شود.

- معی یکی از فرمول‌های چیست؟ یک تکنیک ریاضی است. وقتی حل یک مسئله دشوار است (معادلات) گاهی

بسیار است، قبل از شروع حل، آن مسئله را در یک ماتریس گسلی ضرب ضرب می‌کنند تا محاسبات را تسهیل

شود و کامپیوتر آن را سریع‌تر حل کند. مثلاً یک بردار عمودی و یک بردار عمودی را تصور کنید. این بردارها با نگاه کردن

به یک زمین (کارت بازی) به یک دره می‌بینیم، و یک از یک دره به یک دره می‌بینیم و یک از یک دره به یک دره می‌بینیم.

که منجر به گسلی می‌شود. می‌توانیم به یک بردار عمودی نگاه کنیم و یک بردار عمودی را به یک بردار عمودی

می‌بینیم (مثلاً در یک بردار عمودی). با این تغییر فضای هندسی، می‌توانیم به یک بردار عمودی نگاه کنیم و یک بردار عمودی

و می‌توانیم به یک بردار عمودی نگاه کنیم. این کار با ضرب کردن معادلات در یک ماتریس گسلی (ماتریس یک بردار عمودی) انجام می‌شود.

- روی کارهای من چیست؟ این روی یک روشی است که می‌تواند برای حل دستگاه معادلات غیر خطی

بسیار ساده است. در این روش، به جای حرکت در جهت‌های تصادفی، در هر مرحله در جهتی حرکت می‌کنیم

که نسبت به جهت‌های قبلی متغیر اطلاعات ضروری باشد تا می‌توانیم به جواب گویا تر برسیم.

الف) درستی یکای روی‌های شبه نیوتونی

برای بررسی درستی یکای فروغ می‌نیم واحد متغیر را برابر با $[X]$ و واحد تابع هدف $f(x)$

برابر با $[F]$ باشد. در این صورت، یکای کت‌های مختلف به شرح زیر است:

گرادیان: $g = \nabla f(x)$ دارای یکای $\frac{[F]}{[X]}$ است.

هسین: $H = \nabla^2 f(x)$ دارای یکای $\frac{[F]}{[X]^2}$ است.

تقریب معکوس هسین: $B \approx H^{-1}$ دارای یکای $\frac{[X]^2}{[F]}$ است.

گام: $s_k = \alpha_k \frac{\alpha_k}{\alpha_{k+1}}$ دارای یکای $[X]$ است.

تفاضل گرادیان: $y_k = g_{k+1} - g_k$ دارای یکای $\frac{[F]}{[X]}$ است.

در روی‌های شبه نیوتونی، معادله سکانت (Secant Equation) به صورت $B_{k+1} y_k = s_k$ است.

بررسی یکای این معادله: $[B] \times [y] = \left(\frac{[X]^2}{[F]} \right) \times \left(\frac{[F]}{[X]} \right) = [X]$ که صحیحاً برابر با یکای s_k است.

حالا باید فرمول به روز رسانی د ف ب برابر معکوس هسین را بررسی کنیم: $B_{k+1} = B_k + \frac{s_k s_k^T}{y_k^T s_k} - \frac{B_k y_k y_k^T B_k}{y_k^T B_k y_k}$

حال باید یکای عملیات را بررسی کنیم. جمله اول B_k جمله اول $\frac{[X]^2}{[F]}$

جمله دوم $\left(\frac{s_k s_k^T}{y_k^T s_k} \right)$: کسر در صورت دارای یکای $[X]^2$ و درخرج دارای یکای $[F]$ است $\left(\frac{[F]}{[X]} \right) \times [X] = [F]$

پس کل کسر یکای $\frac{[X]^2}{[F]}$ دارد. جمله سوم $\left(\frac{B_k y_k y_k^T B_k}{y_k^T B_k y_k} \right)$: در صورت، یکای $[X]$ دارد، پس

صورت کسر $[X] \times [X] = [X]^2$ است. درخرج $y_k^T B_k y_k$ یکای $[F]$ دارد. پس کل کسر یکای $\frac{[X]^2}{[F]}$ دارد.

بنابراین همه عملیات یکای هم دارند و فرمول ما از نظر ابعادی کاملاً سازگار درست هستند. این برای BFGS هم صادق است.

ب) معادله خانواده بدوین و تبدیل آن به د ف و ب ف می باشد.

خانواده بدوین یک علامت کمی از فرمول های بدوین برای تقریب محلی (B_k) است که ترکیبی

خطی از بدوین های د ف و ب ف می باشد. (از آنجمله فرمول های آن یک پارامتر ϕ به

$$B_{k+1}^{\phi} = (1 + \phi) B_{k+1}^{DFD} + \phi B_{k+1}^{BFGS} \quad \text{نمی توانیم تعیین کنیم}$$

$$B_{k+1}^{\phi} = B_{k+1}^{DFD} + \phi (B_{k+1}^{BFGS} - B_{k+1}^{DFD}) \quad \text{یا به فرم دیگر:}$$

حال این معادله را با دو مقدار متداول ϕ امتحان کنیم:

$$\textcircled{1} \text{ اگر } \phi = 0 \text{ مقدار دهیم: } B_{k+1}^{\phi=0} = (1-0) B_{k+1}^{DFD} + 0 \times B_{k+1}^{BFGS} \Rightarrow B_{k+1} = B_{k+1}^{DFD}$$

در این حالت، معادله مستقیماً به روش د ف تبدیل می شود (بدوین - تغییر - جدول) تبدیل می شود.

$$\textcircled{2} \text{ اگر } \phi = 1 \text{ مقدار دهیم: } B_{k+1}^{\phi=1} = (1-1) B_{k+1}^{DFD} + 1 \times B_{k+1}^{BFGS} \Rightarrow B_{k+1} = B_{k+1}^{BFGS}$$

در این حالت، معادله به طور مستقیم به روش ب ف تبدیل می شود. این موضوع نشان می دهد که

د ف و ب ف یک دو حالت خاص در بین خانواده $\phi \in [0, 1]$ از خانواده بدوین

بدوین هستند.

ج) ارتباط روسی های شبه نیوتونی با پیک سرهای سازش ها در لایان مزدوج

در روسی لایان مزدوج پیک سر طی شده برای حل دستگاه معادلات $Ax=b$ (۱) بین سازش تابع

درجه دوم (متناظر)، ما از یک ماتریس پیک سر M استفاده می کنیم که تقریب خوبی از ماتریس A هست

باشد ($M \approx A$) τ سر طی شده منتهی به $Mz=r$ در عدد کدر سیستم (که r باقی مانده لایان منفی است)

حل هر مورد، یعنی جهت حرکت مناسب با $M^{-1}r$ است.

همه ارتباط این موضوع با روسی های شبه نیوتونی در مورد زیر گفته است:

۱) ماتریس B_k به عنوان پیک سر ط سازش در روسی های شبه نیوتونی، جهت جستجو به صورت $(B_k - B_k) \nabla f(x_k) = P_k$

عاب هر مورد در اینجا B_k نقش همان ماتریس M^1 (معکوس پیک سر ط سازش) را بازی می کند. با ندرت تغییرات

B_k تقریب بهتر از معکوس H^1 هر مورد و به طور پیک سر ط سازش تقریبی را اینها می کنند.

۲) توالی محبات مزدوج: ماتر شده است که اگر توالی هدف درجه دوم باشد و جستجوی محلی دقیق اینها

شود، روسی های شبه نیوتونی باشد و P و B یک (با فرض شروع از B_0) دنباله محبات

جستجو را توالی می کنند که روسی لایان مزدوج است و دارد تولید می کنند.

۳) روسی های شبه نیوتونی بدون محافظه: روسی های شبه نیوتونی با محافظه محدود از نظر سازش بسیار

پیکر به روسی لایان مزدوج پیک سر برای شده با محافظه محدود هستند. در واقع، هر توالی لایان مزدوج

را یک محبات خاص از روسی شبه نیوتونی در نظر اندازیم که ماتریس تقریب H را در ذهنه نمی شود.

الگوریتم L_BFGS یا بـفـگـش-حافظه محدود را بنویسید و توضیح دهید. ب- همان‌گونه که گرادیان نزولی تصادفی در درس معرفی شد بـفـگـش نیز دارای نسخه‌ای تصادفی برای حل مسائل کوژ با نام اختصاری O-BFGS بـفـگـش-برخط است. آن را توضیح دهید. روی مجموعه داده‌ای که با تابع هزینه کمترین مربعات اجرا کنید و نتایج را با بـفـگـش معمول مقایسه کنید.

الف) بررسی و توضیح الگوریتم L-BFGS (بـفـگـش - حافظه محدود)

برای درک منطق L-BFGS، ابتدا باید نقطه ضعف الگوریتم پایه یعنی بـفـگـش را بررسی کنیم. چون به لحاظ منطقی حتما الگوریتمی مشکلی داشته که برای حل مشکل آن الگوریتم دیگری طراحی شده است. می‌دانیم که روش‌های شبه نیوتنی در هر تکرار k ، برای یافتن جهت حرکت p_k ، از تقریب معکوس ماتریس هسین استفاده می‌کنند. اگر این ماتریس تقریب زده شده را با H_k نشان دهیم، جهت حرکت از رابطه زیر به دست می‌آید:

$$p_k = -H_k \nabla f(x_k)$$

مشکل اساسی وقتی خودش را نشان می‌دهد که با مسائل بهینه سازی مقیاس بزرگ مانند آموزش شبکه‌های عصبی عمیق یا مدل‌هایی با میلیون‌ها ویژگی مواجه می‌شویم. مثلاً بیایید فرض کنیم بردار ورودی ما $x \in \mathbb{R}^n$ باشد و $n = 1,000,000$ باشد. در این حالت، ماتریس H_k یک ماتریس متراکم (Dense) با ابعاد $n \times n$ خواهد بود.

ذخیره چنین ماتریسی در حافظه سیستم نیازمند $O(n^2)$ واحد حافظه است (یعنی انگار برای یک میلیون متغیر، به ترابایت‌ها رم نیاز داریم). علاوه بر این، به روزرسانی این ماتریس عظیم در هر تکرار، هزینه محاسباتی بسیار بالایی دارد. بنابراین همین مشکل سبب متولد شدن الگوریتم L-BFGS (Limited-memory BFGS) برای حل این بن بست شده است.

ایده اصلی L-BFGS این است که تاریخچه قدیمی را دور بی‌اندازیم. به طوری که می‌گویید ما اصلاً نیازی نداریم ماتریس عظیم H_k را به صورت صریح در حافظه ذخیره کنیم. در الگوریتم BFGS استاندارد، ماتریس H_{k+1} با استفاده از ماتریس مرحله قبل H_k و دو بردار به‌روز رسانی ساخته می‌شود:

- بردار تغییرات مکان: $s_k = x_{k+1} - x_k$

- بردار تغییرات گرادیان: $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$

الگوریتم L-BFGS به جای ذخیره کل ماتریس متراکم، تنها تعداد محدودی از بردارهای s و y در چند گام اخیر را در حافظه نگه می‌دارد. ما یک عدد ثابت مثل m (اغلب بین ۵ تا ۲۰) تعریف می‌کنیم. الگوریتم فقط m جفت بردار اخیر یعنی $\{(s_i, y_i)\}^{k-1}$, $i=k-m$ را ذخیره می‌کند و بردارهای قدیمی‌تر را دور می‌اندازد. با این کار، نیاز به حافظه از $O(n^2)$ به $O(mn)$ کاهش می‌یابد که یک صرفه جویی واقعا جالب است.

حالا زمان فرمول بندی ریاضی و نحوه محاسبه بدون تشکیل ماتریس فرا رسیده است. در روش استاندارد، فرمول به‌روزرسانی معکوس هسین به شکل زیر است:

$$H_{k+1} = V_K^T H_k V_k + \rho_k s_k s_k^T$$

که در آن $V_k = I - \rho_k y_k s_k^T$ و $\rho_k = \frac{1}{y_k^T s_k}$ است (ا ماتریس همانی است).

در L-BFGS، ما هیچ وقت این ماتریس را تشکیل نمی‌دهیم. هدف ما تنها محاسبه ضرب ماتریس در بردار گرادیان، یعنی محاسبه مقدار $H_k \nabla f(x_k)$ برای یافتن جهت حرکت است. برای این کار، فرض می‌کنیم یک تقریب اولیه خام برای معکوس هسین داریم که در بیشتر موارد آن را یک ضربی از ماتریس همانی در نظر می‌گیرند:

$$H_k^0 = \gamma_k I$$

که ضریب مقیاس‌پذیری معمولاً به این شکل محاسبه می‌شود:

$$\gamma_k = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}$$

حالا با داشتن این ماتریس اولیه و m جفت بردار ذخیره شده، برای محاسبه $p_k = -H_k \nabla f(x_k)$ از یک الگوریتم بسیار معروف و کارآمد به نام الگوریتم بازگشتی دو حلقه‌ای (two-loop recursion) استفاده می‌کنیم.

حالا زمان آن فرا رسیده که روند کار الگوریتم بازگشتی دو حلقه‌ای را که نقش مهمی در ب‌ف‌گ‌ش با حافظه محدود دارد. این الگوریتم نشان می‌دهد که چگونه فقط با ضرب داخلی بردارها (بدون هیچ ضرب ماتریسی) می‌توانیم جهت بهینه را پیدا کنیم. روند کار به این شکل است:

گام اول: باید مقداردهی اولیه را انجام دهیم

$$q = \nabla f(x_k)$$

گام دوم: حلقه اول را داریم و رو به عقب در تاریخچه حرکت می‌کنیم.

برای $i = k-1, k-2, \dots, k-m$:

$$\alpha_i = \rho_i s_i^T q$$

$$q = q - \alpha_i y_i$$

گام سوم: حالا باید در ماتریس تقریب اولیه تقسیم کنیم.

$$z = H_k^0 q = \gamma_k q$$

گام چهارم: بعد حلقه دوم را داریم و باید در تاریخچه رو به جلو حرکت کنیم:

برای $i = k-m, \dots, k-2, k-1$:

$$\beta = \rho_i y_i^T z$$

$$z = z + s_i (\alpha_i - \beta)$$

گام پنجم: در نهایت نوبت به خروجی میرسد. حالا بردار z به دست آمده، همان حاصل ضرب مد نظر ماست و جهت حرکت برابر خواهد بود با:

$$p_k = -z$$

با تمام این اوصاف، L-BFGS یا همان بـفـگـش با حافظه محدود، با جایگزین کردن یک ماتریس تقریب معکوس هسین بسیار بزرگ با مجموعه‌ای از بردارهای تاریخچه نزدیک و استفاده از یک حلقه محاسباتی هوشمندانه، توانسته دقت همگرایی روش‌های شبه نیوتنی را با مصرف حافظه‌ای در حد روش‌های گرادیان کاهشی ترکیب کند. برای همین هم بـفـگـش با حافظه محدود یکی از قدرتمندترین الگوریتم‌ها برای مسائل مقیاس‌بزرگ به شمار می‌آید.

ب) بررسی و تحلیل الگوریتم O-BFGS نسخه برخط و تصادفی

در ابتدا برای من این سوال ایجاد شد که اصلاً چرا به نسخه برخط نیاز داریم؟ همان‌طور که در بخش قبل دیدیم، بـفـگـش با حافظه محدود مشکل کمبود حافظه را حل کرد؛ اما هنوز یک مشکل بزرگ پابرجاست که هزینه محاسباتی گرادیان در هر تکرار می‌باشد. در مسائل یادگیری ماشین، تابع هزینه اغلب مجموع خطای مدل روی کل داده‌هاست:

$$f(x) = \frac{1}{N} \sum_{i=1}^N f_i(x)$$

در L-BFGS (و BFGS استاندارد)، ما برای محاسبه جهت حرکت، به گرادیان کل داده‌ها $\nabla f(x_k)$ نیاز داریم. اگر تعداد داده‌ها N مثلا ۱۰ میلیون باشد، محاسبه یک مرحله گرادیان بسیار زمان‌بر خواهد بود. اینجا است که ایده روش‌های تصادفی یا برخط مطرح می‌شود؛ یعنی به جای استفاده از کل داده‌ها، در هر مرحله فقط از یک زیرمجموعه کوچک تصادفی از داده‌ها استفاده کنیم. اما پیاده‌سازی این ایده روی روش‌های شبه نیوتنی اصلا کار ساده‌ای نیست.

اما چالش اساسی ب‌ف‌گ‌ش در حالت تصادفی، نویز در بردار y_k است. در روش‌های گرادیان نزولی تصادفی (SGD)، استفاده از مینی‌بچ‌ها یا همان زیرمجموعه کوچک تصادفی به خوبی کار می‌کند. اما وقتی می‌خواهیم ب‌ف‌گ‌ش را تصادفی کنیم با یک بحران ریاضی مواجه می‌شویم. می‌دانیم که در ب‌ف‌گ‌ش، تقریب ماتریس هسین بر اساس بردار تغییرات گرادیان یعنی y_k به روز رسانی می‌شود:

$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$$

اگر بخواهیم این را به صورت تصادفی بنویسیم، گرادیان مرحله k روی مینی‌بچ S_k محاسبه می‌شود و گرادیان مرحله $k+1$ روی یک مینی‌بچ کاملا متفاوت S_{k+1} محاسبه خواهد شد. در نتیجه داریم:

$$y_k = \nabla f_{S_{k+1}}(x_{k+1}) - \nabla f_{S_k}(x_k)$$

این معادله واقعا سخت است. چون اختلاف بین این دو گرادیان، دیگر نشان دهنده انحنای تابع (Curvature) نیست، بلکه بیشتر این اختلاف به دلیل تغییر کردن نمونه‌های داده (تفاوت S_{k+1} با S_k) است. این نویز شدید باعث می‌شود که شرط اساسی ب‌ف‌گ‌ش یعنی $S_k^T y_k > 0$ نقض شود، ماتریس هسین از حالت مثبت معین (Positive Definite) خارج شود و الگوریتم به جای همگرایی، واگرا شود و طبق چیزی که فهمیدیم اصطلاحا منفجر شود.

حالا راهکار ب‌ف‌گ‌ش برخط این است که روی مینی‌بچ‌های یکسان محاسبه انجام دهیم. الگوریتم ب‌ف‌گ‌ش برخط (که توسط محققانی مانند Schraudolph و بعدا Byrd توسعه یافت) برای حل این مشکل یک ایده ساده اما بسیار حیاتی را معرفی کرد که برای محاسبه بردار y_k باید هر دو گرادیان روی یک مینی‌بچ یکسان محاسبه شوند.

یعنی در هر مرحله k ، ما علاوه بر گرادیان معمولی که برای حرکت استفاده می‌کنیم، یک بار دیگر گرادیان نقطه جدید را روی همان داده‌های قبلی حساب می‌کنیم تا بردار y_k بدون نویز تغییر داده‌ها به دست آید.

- روند اجرای الگوریتم بـفـگـش برخط:

الگوریتم در هر تکرار k مراحل زیر را طی می‌کند:

گام اول: اول یک مینی‌بچ تصادفی S_k از داده‌ها انتخاب می‌کنیم.

گام دوم: گرادیان تصادفی را برای نقطه فعلی محاسبه می‌کنیم:

$$g_k = \nabla f_{S_k}(x_k)$$

گام سوم: جهت حرکت را با استفاده از تقریب هسین (که با روش حافظه محدود L-BFGS هم قابل ترکیب است) پیدا می‌کنیم:

$$p_k = -H_k g_k$$

گام چهارم: متغیرها را به روزرسانی می‌کنیم. (البته من فهمیدم که در روش‌های آنلاین اغلب از جستجوی خطی یا Line Search استفاده نمی‌کنیم چون نویز دارد، بلکه از یک نرخ یادگیری یا Step Size کاهش η_k استفاده می‌کنیم):

$$x_{k+1} = x_k + \eta_k p_k$$

گام پنجم: برای یافتن y_k ، گرادیان نقطه جدید x_{k+1} را روی همان مینی‌بچ S_k دوباره حساب می‌کنیم:

$$\tilde{g}_{k+1} = \nabla f_{S_k}(x_{k+1})$$

گام ششم: بردارهای به‌روزرسانی را می‌سازیم و ماتریس H را برای مرحله بعد به روز می‌کنیم:

$$S_k = x_{k+1} - x_k$$

$$y_k = \tilde{g}_{k+1} - g_k$$

بنابراین الگوریتم بـفـگـش برخط با الزام به استفاده از مینی‌بچ‌های یکسان برای محاسبه اختلاف گرادیان‌ها، توانست مشکل نویز را حل کند و اطلاعات انحنای تابع (ماتریس هسین) را به درستی استخراج کند. با این کار، ما سرعت و مقیاس‌پذیری روش‌های تصادفی مثل SGD را با قدرت همگرایی و هوشمندی روش‌های شبه نیوتنی به درستی ترکیب کرده‌ایم.

حالا پس از بررسی مبانی تئوری الگوریتم بـفـگـش برخط و درک نحوه غلبه آن بر چالش نویز در مینی‌بچ‌ها، نوبت به ارزیابی عملی این الگوریتم می‌رسد. برای این منظور، یک مسئله کلاسیک رگرسیون با تابع هزینه کمترین مربعات (least squares) را در نظر گرفته‌ایم. هدف این است که نشان دهیم چگونه انتقال از حالت پردازش

کل داده‌ها (full batch در BFGS استاندارد) به حالت پردازش دسته‌ای (mini batch در O-BFGS) قادر است روی سرعت همگرایی تاثیر بگذارد. در همین راستا کد زیر بدون استفاده از توابع آماده بهینه سازی و صرفاً با جبر خطی پایه در نامپای توسعه داده شده است تا منطق ریاضی الگوریتم‌ها به وضوح نشان داده شود.

توضیح کد:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression

# تولید دیتاست مصنوعی برای تست الگوریتم‌ها
# یک دیتاست با 1000 نمونه و 10 ویژگی می‌سازیم
X, y = make_regression(n_samples=1000, n_features=10,
                       noise=0.5, random_state=42)

# (Bias) برای محاسبه عرض از مبدأ X اضافه کردن ستون یک‌ها به
X = np.c_[np.ones(X.shape[0]), X]
n_samples, n_features = X.shape

# تعریف توابع پایه: تابع هزینه و گرادیان (کمترین مربعات)
def compute_loss(w, X_data, y_data):
    # محاسبه تابع هزینه کمترین مربعات (Least Squares)
    predictions = X_data.dot(w)
    return 0.5 * np.mean((predictions - y_data) ** 2)

def compute_gradient(w, X_data, y_data):
    # محاسبه گرادیان تابع هزینه
    predictions = X_data.dot(w)
    return X_data.T.dot(predictions - y_data) / X_data.shape[0]
```

در بخش اول کد، ابتدا ابزارهای لازم برای محاسبات ریاضی و رسم نمودار را به برنامه اضافه کردم. بعد برای اینکه بتوانیم الگوریتم‌هایمان را امتحان کنیم، یک مجموعه داده فرضی ساختم که هزار نمونه و ده ویژگی دارد تا یک بستر آزمایش آماده و مشخص داشته باشیم. بلافاصله بعد از آن، یک ستون کامل از عدد یک به داده‌هایمان چسبانیدیم؛ این یک ترفند ساده و معروف است که کمک می‌کند مدل بتواند عرض از مبدا را خودش به راحتی یاد بگیرد و درگیر فرمول نویسی جداگانه برای آن نشوم.

در ادامه دو تابع اصلی کارمان را تعریف کردم. تابع اول وظیفه دارد میزان خطای مدل را اندازه بگیرد؛ روش کارش به این صورت است که حدس‌های مدل را با جواب‌های واقعی مقایسه می‌کند، اختلاف آن‌ها را به توان دو می‌رساند تا منفی و مثبت شدن خطاها باعث خنثی شدن هم نشوند و در نهایت میانگین این خطاها را به ما می‌دهد. تابع دوم هم شیب این خطا را حساب می‌کند که مثل یک قطب نما عمل کرده و به الگوریتم نشان می‌دهد برای کمتر شدن اشتباهات، باید اعداد و وزن‌های مدل را دقیقاً به کدام جهت تغییر دهیم.

```

# پیاده‌سازی الگوریتم بیگش معمولی (استاندارد)
def standard_bfgs(X, y, max_iterations=50, lr=0.1):
    w = np.zeros(n_features) # مقداردهی اولیه وزن‌ها
    H = np.eye(n_features) # مقداردهی اولیه ماتریس تقریب هسین
    loss_history = []

    for i in range(max_iterations):
        # ثبت خطای کل در این تکرار
        loss_history.append(compute_loss(w, X, y))
        # محاسبه گرادیان روی کل داده‌ها
        g = compute_gradient(w, X, y)
        # تعیین جهت حرکت
        p = -H.dot(g)
        # به‌روزرسانی وزن‌ها
        w_new = w + lr * p
        # محاسبه گرادیان جدید برای یافتن
        g_new = compute_gradient(w_new, X, y)
        # بردارهای اختلاف
        s = w_new - w
        y_vec = g_new - g
        # به‌روزرسانی ماتریس هسین (فرمول بیگش)
        rho_den = np.dot(y_vec, s)
        if rho_den > 1e-10: # جلوگیری از تقسیم بر صفر و حفظ مثبت معین بودن
            rho = 1.0 / rho_den
            I = np.eye(n_features)
            V = I - rho * np.outer(y_vec, s)
            H = V.T.dot(H).dot(V) + rho * np.outer(s, s)
        w = w_new
    return loss_history

```

در این بخش، نسخه معمولی و پایه الگوریتم را پیاده‌سازی کردیم. در شروع کار، برای اینکه نقطه آغازی داشته باشیم، تمام وزن‌های مدل را صفر در نظر گرفتیم و ماتریس تقریب (که مثل حافظه الگوریتم عمل می‌کند و انحنای مسیر را در خود نگه می‌دارد) را با یک ماتریس خنثی و ساده مقداردهی کردیم. همچنین یک فضای خالی ساختیم تا در هر مرحله، میزان خطای مدل را در آن ثبت کنیم و بعداً بتوانیم روند یادگیری را روی نمودار بکشیم. در ادامه وارد حلقه اصلی الگوریتم می‌شویم. در هر تکرار، مدل اول خطای خودش را می‌سنجد و ثبت می‌کند. سپس با نگاه کردن به تمام داده‌ها به صورت یکجا، شیب مسیر را حساب کرده و با کمک حافظه‌ای که از انحنای مسیر دارد، بهترین جهت برای پایین رفتن و کم کردن خطا را پیدا می‌کند. وقتی مدل در این جهت یک قدم برداشت و وزن‌هایش تغییر کرد، ما شیب مسیر را در این مکان جدید دوباره اندازه می‌گیریم.

حالا به مهم‌ترین قسمت می‌رسیم؛ ما اختلاف بین مکان قبلی و جدید و همچنین اختلاف بین شیب قدیم و جدید را حساب می‌کنیم. قبل از اعمال فرمول‌های پیچیده، با یک شرط ساده ریاضی مطمئن می‌شویم که مخرج کسر صفر نشود و مسیر الگوریتم خراب نشود. اگر این شرط برقرار بود، با استفاده از همان اختلاف‌هایی که حساب کردیم، حافظه مدل (ماتریس تقریب) را به‌روز می‌کنیم تا برای قدم بعدی دقیق‌تر عمل کند و در نهایت با این وزن‌های جدید به استقبال مرحله بعدی می‌رویم.

```

# نسخه برخط 0-BFGS پیاده‌سازی الگوریتم
def online_bfgs(X, y, epochs=50, batch_size=32, lr=0.1):
    w = np.zeros(n_features)
    H = np.eye(n_features)
    loss_history = []

    for epoch in range(epochs):
        # ثبت خطا در ابتدای هر اپوک (برای مقایسه عادلانه با روش استاندارد)
        loss_history.append(compute_loss(w, X, y))
        # بر هم زدن تصادفی داده ها در هر اپوک
        indices = np.random.permutation(n_samples)
        # حرکت روی مینی بچ ها
        for i in range(0, n_samples, batch_size):
            batch_idx = indices[i:i+batch_size]
            X_batch, y_batch = X[batch_idx], y[batch_idx]

            # محاسبه گرادیان روی مینی بچ
            g = compute_gradient(w, X_batch, y_batch)
            p = -H.dot(g)
            # نرخ یادگیری کاهشی (در روش‌های آنلاین بهتر جواب می دهد)
            step_size = lr / (1 + 0.05 * epoch)
            w_new = w + step_size * p
            # محاسبه گرادیان جدید روی همان مینی‌بچ قبلی
            g_new_same_batch = compute_gradient(w_new, X_batch, y_batch)
            s = w_new - w
            y_vec = g_new_same_batch - g # محاسبه y تغییر مینی بچ
            # به‌روزرسانی ماتریس هسین
            rho_den = np.dot(y_vec, s)
            if rho_den > 1e-8: # شرط انحنا
                rho = 1.0 / rho_den
                I = np.eye(n_features)
                V = I - rho * np.outer(y_vec, s)
                H = V.T.dot(H).dot(V) + rho * np.outer(s, s)
            w = w_new
    return loss_history

```

در این بخش نسخه برخط یا همان آنلاین الگوریتم را پیاده‌سازی کردیم. شروع کار دقیقاً مثل روش قبلی است؛ یعنی نقطه‌ها را صفر می‌کنیم و یک حافظه خام و یک لیست خالی برای ثبت خطا می‌سازیم. تفاوت اصلی از اینجا شروع می‌شود که در هر دوره مرور داده‌ها، اول از همه کل داده‌ها را مثل یک دسته کارت بُر می‌زنیم تا ترتیبشان کاملاً تصادفی شود و مدل نتواند ترتیب خاصی را حفظ کند.

سپس به جای اینکه کل داده‌ها را یکجا به مدل نشان دهیم، آن‌ها را به دسته‌های خیلی کوچک‌تری تقسیم می‌کنیم. مدل روی هر دسته کوچک شیب مسیر را حساب کرده و جهت حرکت را پیدا می‌کند. یک نکته جالب اینجا این است که اندازه قدم‌های مدل را طوری تنظیم کردیم که هر چه به پایان کار نزدیک‌تر می‌شویم، قدم‌های کوچک‌تری بردارد تا مدل در نزدیکی هدف دچار لرزش و نوسان نشود و به اشتباه از نقطه مطلوب رد نشود.

در نهایت به مهم‌ترین ترفند این روش می‌رسیم؛ وقتی مدل یک قدم برمی‌دارد و به نقطه جدیدی می‌رسد، ما شیب این نقطه جدید را دقیقاً روی همان دسته کوچک قبلی حساب می‌کنیم. این کار بسیار حیاتی است چون باعث می‌شود نویز و خطای ناشی از عوض شدن دسته‌های داده از بین برود و مدل اصلاً گیج نشود. بعد از آن هم

مثل روش معمولی، با چک کردن یک شرط ایمنی ریاضی، حافظه مدل را به روز می‌کنیم تا برای گرفتن دسته بعدی داده‌ها آماده و دقیق‌تر باشد.

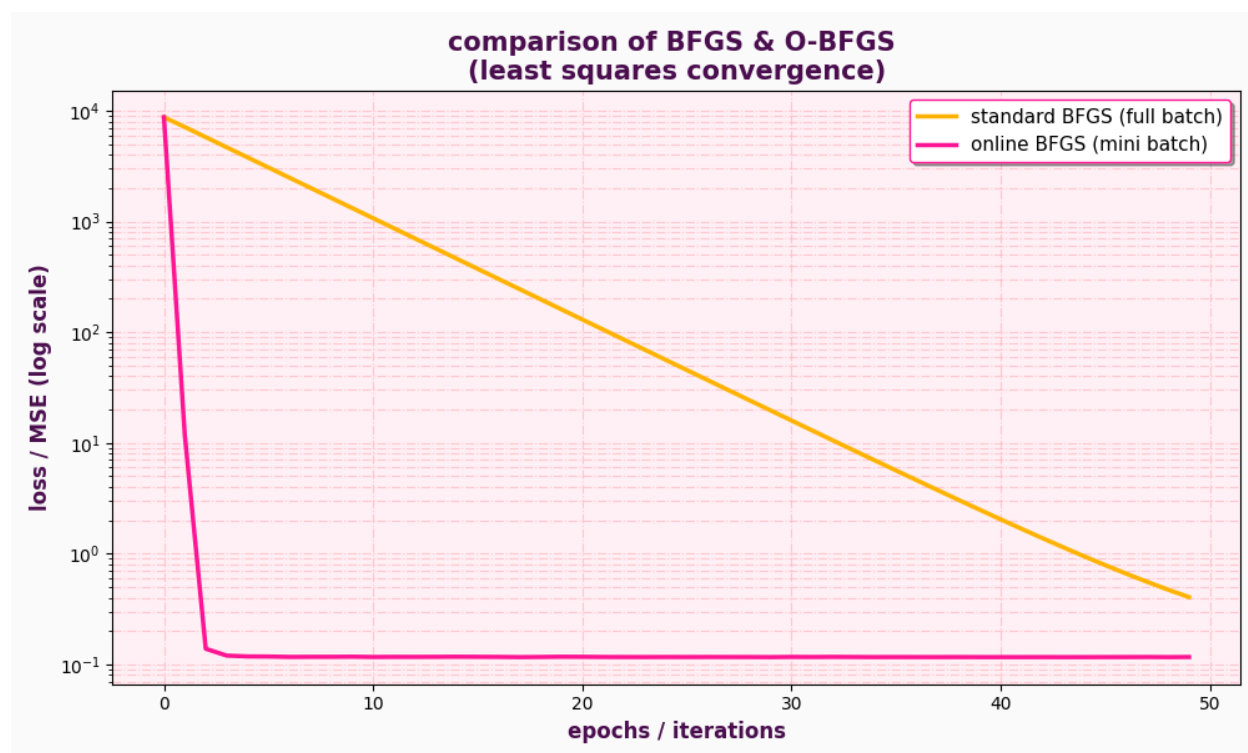
```
# اجرا و رسم نمودار مقایسه
print("running standard BFGS...")
bfgs_history = standard_bfgs(X, y, max_iterations=50)
print("running online BFGS (O-BFGS)...")
obfgs_history = online_bfgs(X, y, epochs=50, batch_size=32)
# تنظیمات اصلی فیکور و پس مینه
fig, ax = plt.subplots(figsize=(10, 6))
fig.patch.set_facecolor('FAFAFA')
ax.set_facecolor('FFF0F5') # رنگ پس زمینه
# استاندارد BFGS رسم خط
plt.plot(bfgs_history, label='standard BFGS (full batch)',
         color='FFB300', linewidth=2.5, markersize=10, markeredgecolor='FF8F00')
# رسم خط O-BFGS
plt.plot(obfgs_history, label='online BFGS (mini batch)',
         color='FF1493', linewidth=2.5, markersize=11, markeredgecolor='C71585')
# استایلدهی به عنوان و محورها
plt.title('comparison of BFGS & O-BFGS \n(least squares convergence)',
         fontsize=15, fontweight='bold', color='4A0E4E')
plt.xlabel('epochs / iterations', fontsize=12,
         fontweight='bold', color='4A0E4E')
plt.ylabel('loss / MSE (log scale)', fontsize=12,
         fontweight='bold', color='4A0E4E')
# تنظیم مقیاس و گزید لاین‌ها
plt.yscale('log')
plt.grid(True, which="both", linestyle="-.", color='FFB6C1', alpha=0.7)
# استایلدهی به راهنمای نمودار
legend = plt.legend(fontsize=11, loc='upper right', frameon=True, shadow=True)
legend.get_frame().set_facecolor('FFFFFF')
legend.get_frame().set_edgecolor('FF1493')
# نمایش نهایی
plt.tight_layout()
plt.show()
```

در این بخش نهایی، وقت آن است مقایسه این دو روش را ببینیم. اول از همه، هر دو الگوریتم (نسخه معمولی و نسخه برخط) را به طور واقعی روی داده‌ها اجرا می‌کنیم. به هر دو اجازه می‌دهیم ۵۰ مرحله کارشان را انجام دهند تا یک مسابقه کاملاً عادلانه شکل بگیرد و رکوردهای خطای هر کدام را در طول مسیر برای رسم نمودار ذخیره می‌کنیم.

در قدم بعدی، رکوردهای ثبت شده را می‌کشیم. برای نشان دادن مسیر الگوریتم معمولی از یک خط طلایی پررنگ و برای روش آنلاین از یک خط صورتی تیره استفاده کردم. ضخامت خطوط را هم کمی بیشتر می‌کنم تا دیده شوند. همچنین برای نمودارمان یک عنوان مشخص می‌نویسیم و برای محورهای افقی (تعداد مراحل) و عمودی (میزان خطا) اسم می‌گذاریم و رنگ متن‌ها را بنفش تیره کردم.

یک ترند بسیار مهم که در اینجا استفاده کردم، تغییر مقیاس محور عمودی به حالت لگاریتمی است. به زبان ساده، چون خطای مدل به سرعت سقوط می‌کند و خیلی زود به اعداد بسیار کوچکی می‌رسد، در یک نمودار عادی دیدن جزئیات این خطاهای کوچک در اواخر مسیر تقریباً غیرممکن است. این مقیاس خاص مثل یک ذره

بین عمل می‌کند تا بتوانیم افت خطا را در تمام مراحل خیلی واضح‌تر ببینیم. در نهایت، با اضافه کردن خطوط راهنمای شبکه‌ای کم رنگ در پس زمینه و قرار دادن یک کادر راهنما در گوشه تصویر که معرفی می‌کند هر رنگ مربوط به کدام روش است، این نمودار را روی صفحه نمایش می‌دهیم.



این نمودار نتیجه مقایسه بین دو الگوریتم مذکور است؛ محور پایین تعداد مراحل طی شده و محور کناری میزان خطای مدل را نشان می‌دهد. خط طلایی (الگوریتم استاندارد) را ببینید که چطور با سرعتی ثابت و ملایم در طول ۵۰ مرحله خطا را کم می‌کند، چون در هر قدم مجبور است تمام داده‌ها را یکجا پردازش کند. اما در طرف مقابل، خط صورتی (الگوریتم برخط) قرار دارد که با استفاده از دسته‌های کوچک داده، در همان یکی دو مرحله‌ی اول با یک سقوط آزاد خطایش به حداقل ممکن رسیده و همان پایین ثابت مانده است. این تصویر به ساده‌ترین شکل ممکن ثابت می‌کند که خرد کردن داده‌ها و برداشتن قدم‌های سریع و پی در پی در روش آنلاین، چقدر می‌تواند سرعت یادگیری و رسیدن به کمترین خطا را نسبت به روش‌های سنتی و کند افزایش دهد.

با این حساب تلفیق ایده مینی بیچ‌ها با روش‌های شبه نیوتنی در الگوریتم ب‌ف‌گ‌ش برخط باعث می‌شود که در مسائل مقیاس‌بزرگ یادگیری ماشین، بتوانیم با هزینه محاسباتی بسیار کمتر، از اطلاعات انحنای تابع نیز بهره‌مند شویم و به سرعت همگرایی فوق‌العاده‌ای در مراحل اولیه دست یابیم.

منابع:

منابعی که من در انجام این تمرین برای هر سوال به کار گرفتم، شامل موارد زیر هستند:

سوال یک:

من برای این تمرین همان منابع ذکر شده در تمرین آخر سری اول تمارین بهینه‌سازی محدب را مورد استفاده قرار دادم. ترجیح من این بود که از دوباره‌گویی پرهیز کنم، اما برای تکمیل درست منابع و مجزا بودن این سری از تمرین، همان منابع را دوباره عنوان می‌کنم:

- تعریف تابع روزن بروک و آشنایی با مفهوم آن
- برای درک بیشتر تابع روزن بروک و شکل آن، از جستجوگر ذره‌بین استفاده کردم، متأسفانه الان دقیقاً لینک‌هایی که هنگام جستجو باز کردم را پیدا نکردم و برای جلوگیری از پیوست منبع نامرتبط جستجویم در ذره بین را قرار می‌دهم.
- تعریف گرادیان نزولی و مفهوم آن را از دکتر رزاقی در جلسه اول یادگیری ماشین یاد گرفتم.
- مباحث کمینه محلی و سراسری در صفحات ۲-۱۴ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی دکتر هوشمند
- مفهوم و اهمیت طول قدم در صفحات ۳۳-۳۵ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی دکتر هوشمند
- روش نیوتون صفحات ۵۵ تا ۶۵ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی - با اینکه این روش پس از روش‌های دیگر در جزوه آمده است ولی من در همان ابتدا برای یادگیری مفهوم نیوتون به این بخش مراجعه کردم چون پایه کار دو الگوریتم گرادیان نزولی و نیوتون بودند و سایر الگوریتم‌ها در جایگاه تعیین طول قدم به شمار می‌روند.
- برای درک بیشتر جزوه دکتر هوشمند و تعاریف لازم از مدل زبانی گپ جی پی تی بسیار کمک گرفتم که به علت قطعی اینترنت بین‌المللی برای من خیلی خوب پاسخگو نبود، اما گاهی در درک بهتر و روشن کردن مسیر برایم مفید واقع می‌شد.
- مفهوم شرط ولف به عنوان یک روش برای تعیین طول قدم در صفحات ۳۶-۳۸ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی دکتر هوشمند
- شرط گلدشتاین صفحه ۴۳ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی
- طول قدم با مومنوم در صفحه ۴۷ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی
- طول قدم با نستروف در صفحه ۴۸ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی
- طول قدم با آداگرا در صفحات ۴۹ و ۵۰ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی دکتر هوشمند
- طول قدم با آر ام اس پراپ صفحه ۵۱ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی
- طول قدم با آدالتا صفحه ۵۲ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی
- طول قدم با آدام صفحه ۵۳ جزوه بهینه‌سازی محدب، فصل ۱، مباحث بهینه نامقید، جستجو خطی، گرادیان نزولی

- برای برنامه نویسی و کد این بخش از تمرین سری اول، از جزوه‌های برنامه نویسی دکتر شیرازی که فصل پاییز سال گذشته ارائه داده شده بسیار کمک گرفتم، همچنین بخشی از آموزش و کدهای ماشین لرنینگ که مربوط به گرادیان نزولی بود برای من مفید واقع شد، به سبب راحت تر شدن کدها از ایده ماژولار کد زدن استفاده کردم که دکتر شیرازی به ما آن را آموخته بودند.
- من برای حل این سوال، به علت مرور بهتر و درک بیشتر جزوه، به علاوه رسیدن به سوالات ذهنیم از طریق گپ جی پی تی، توانستم بهتر عمل کنم و متأسفانه به علت قطعی شدید اینترنت، منبع معتبر بیشتری برای ارائه ندارم که از آن استفاده کرده باشم.

سوال دو:

- پاسخ من بر اساس مفاهیم و پایه کتاب‌های مرجع بهینه سازی عددی نوشته شد. طبق چیزی که فهمیدم مشهورترین مرجع جهانی برای این مباحث، کتاب numerical optimization نوشته Nocedal و Wright (فصل‌های مرتبط با روش‌های شبه نیوتونی و گرادیان مزدوج) است. اثبات‌های مذکور در متون درس آمده. البته من برای فهم بیشتر اثبات‌ها و تعاریف اولیه از جزوه خود استاد (قسمت شبه نیوتونی) و گپ جی پی تی کمک گرفتم.
- تعریف گرادیان مزدوج، الگوریتم، روش‌ها و به خصوص مثال‌های صفحات ۳۰-۳۴ از جزوه دکتر هوشمند – فایل دوم از فصل یک؛ مبانی بهینه‌سازی نامقید، گرادیان مزدوج
 - روش نیوتون صفحه ۵ از جزوه دکتر هوشمند – فایل سوم از فصل یک؛ مبانی بهینه‌سازی نامقید، روش‌های نیوتون و شبه نیوتون
 - روش شبه نیوتون صفحه ۶-۱۳ از جزوه دکتر هوشمند – فایل سوم از فصل یک؛ مبانی بهینه‌سازی نامقید، روش‌های نیوتون و شبه نیوتون
 - تعریف روش‌های جستجو خط در صفحه ۲۲ همین جزوه
 - تعریف و روش خانواده برویدن صفحه ۲۵-۲۷ از جزوه دکتر هوشمند – فایل سوم از فصل یک؛ مبانی بهینه‌سازی نامقید، روش‌های نیوتون و شبه نیوتون

سوال سه:

- کتاب numerical optimization نوشته Jorge Nocedal و Stephen J. Wright واقعا برای بهینه سازی مرجع قابل قبول و فوق‌العاده‌ای است که برای سوال ۲ هم برای من مفید بود. جالب است بدانید آقای Nocedal خودش یکی از سازندگان اصلی الگوریتم ب‌ف‌گ‌ش با حافظه محدود است. من برای حل این تمرین از فصل ۷ (Large Scale Unconstrained Optimization) استفاده کردم که به طور کامل به ب‌ف‌گ‌ش بر خط پرداخته است.
- برای درک مفهوم ب‌ف‌گ‌ش برخط و خواسته بخش ب سوال، از گپ جی پی تی استفاده کردم که برخلاف تلاش‌های قبلی، این بار جواب‌های خوبی به من می‌داد و در راهنمایی ام بسیار مفید واقع شد، چون من جزوه این بخش را نداشتم و متأسفانه خیلی خوب نتوانستم مباحث مربوطه را بخوانم، به همین دلیل این مدل زبانی برای یادگیری و حل این بخش از تمرین مفید بود.